Підготовка на Mac та Контекст роботи (ЛР 1)

Все, що треба згадати і запустити перед захистом

🚀 ШПАРГАЛКА ПІДГОТОВКИ ТА КОНТЕКСТ (ДЛЯ ТЕБЕ НА MAC)

Контекст того, як зроблена робота: Ти порівнював дві споріднені UNIX-системи: **macOS 27.2** (на твоєму MacBook Pro Apple M1 Pro, 10 ядер, 16 ГБ ОЗП) та **FreeBSD 15.1-RELEASE arm64** (запускалася у віртуальній машині `QEMU` з прискоренням Apple Hypervisor.framework `-accel hvf`). Звіт відкритий у PDF, а всі macOS-команди можна показати наживо за 1 секунду.

- **1. Автоматичний запуск всього одною командою в Терміналі (macOS):** `cd "/Users/taras/Desktop/unik/Operation Systme" && ./start_defense.sh 1` *Цей скрипт сам відкриє PDF-звіт ЛР1, папку зі скріншотами, запустить програми «Моніторинг системи» і «Дискова утиліта», а в терміналі виконає всі 16 команд macOS зі звіту!*
- **2. Які програми мають бути відкриті на Mac:**
  - **Preview (Оглядач):** файл `lab1/Звіт_ЛР1_Порівняння_ОС_Загорський_Тарас.pdf`
  - **Термінал (macOS):** з виконаним прогоном `./start_defense.sh 1`
  - **Activity Monitor (Моніторинг системи):** вкладка «Пам'ять» (Memory) — щоб показати стиснуту пам'ять (Compressed).
  - **Disk Utility (Дискова утиліта):** щоб показати контейнер APFS `disk3` і томи Macintosh HD / Data.
- **3. Що казати, якщо попросять показати FreeBSD наживо:** Образ FreeBSD 15.1 запускався у QEMU для зняття скріншотів (рис. 8–13) і потім був видалений, щоб звільнити місце під образи Windows 11 (20 ГБ) та Ubuntu (10 ГБ) для наступних лаб. Усі виводи збережені у високій якості на скріншотах `08_FreeBSD_fastfetch.png` – `13_FreeBSD_пакети_pkg.png`.

💻 ПОЯСНЕННЯ КОДУ, ФУНКЦІЙ ТА ПАРАМЕТРІВ

```
cd "/Users/taras/Desktop/unik/Operation Systme" && ./start_defense.sh 1
```

Повний автопрогон ЛР1: відкриває PDF, GUI-утиліти та виконує всі команди macOS у терміналі.

20 Построковий розбір усіх 20 команд звіту (Шпаргалка)

Ключові команди macOS та FreeBSD, їх функції та параметри

ЩО КАЗАТИ ВИКЛАДАЧУ (СЦЕНАРІЙ ДОПОВІДІ)

У практичній частині звіту я використав рівно 20 діагностичних команд UNIX (13 у macOS та 7 у FreeBSD), щоб на низькому рівні порівняти архітектуру ядра, процеси, файлові системи APFS і ZFS, а також підсистеми безпеки.

ПОЯСНЕННЯ КОДУ, ФУНКЦІЙ ТА ПАРАМЕТРІВ

1. `sw_vers`

Функція: виводить назву, версію та номер збірки macOS. **Результат:** macOS 27.2 (Build 26B123).

2. `uname -a`

Функція: виводить інформацію про ядро ОС. **Параметр:** `-a` (all — повний вивід: назва ядра Darwin 27.2.0, версія XNU `xnu-13432.41.4`, архітектура `arm64`).

3. `sysctl kern.ostype kern.osrelease`

Функція: читає параметри стану ядра з MIB-дерева. **Параметри:** `kern.ostype` (тип ядра = Darwin), `kern.osrelease` (реліз = 27.2.0).

4. `sysctl -n machdep.cpu.brand_string`

Функція: повертає назву моделі процесора. **Параметри:** `-n` (не виводити назву ключа, тільки значення), `machdep.cpu.brand_string` (Apple M1 Pro).

5. `sysctl hw.ncpu hw.memsize`

Функція: апаратні характеристики. **Параметри:** `hw.ncpu` (кількість логічних ядер = 10), `hw.memsize` (обсяг ОЗП у байтах = 17179869184, тобто 16 ГБ).

6. `ps -axco pid,user,%cpu,%mem,comm | head -12`

Функція: список запущених процесів. **Параметри:** `-a` (усіх користувачів), `-x` (включно з фоновими демонами без термінала), `-c` (тільки ім'я виконуваного файлу), `-o` (формат колонок). Доводить, що PID 1 — `launchd`.

7. `kmutil showloaded --list-only`

Функція: менеджер розширень ядра (Kernel Management Utility). **Параметри:** `showloaded` (показати завантажені в ядро kext/kpi), `--list-only` (компактний список без залежностей). Показує три складові XNU: `kpi.mach`, `kpi.bsd`, `kpi.iokit`.

8. `launchctl list | head -8`

Функція: керування демонами та агентами підсистеми `launchd`. **Параметр:** `list` (виводить таблицю PID, код завершення Status та назву служби Label).

9. `diskutil list internal`

Функція: утиліта керування дисками macOS. **Параметри:** `list` (схема розділів), `internal` (тільки внутрішній SSD). Показує фізичний диск `disk0` і синтезований контейнер APFS `disk3`.

10. `mount | grep -E 'apfs'`

Функція: виводить таблицю змонтованих файлових систем і фільтрує APFS. Показує прапорці системного тому: `sealed` (криптографічно запечатаний) та `read-only` (лише читання).

11. `df -h / /System/Volumes/Data`

Функція: оцінка вільного місця (disk free). **Параметр:** `-h` (human-readable — у гігабайтах Gi). Доводить спільний пул вільного місця (82 Gi) для кореня `/` і тому даних.

12. `csrutil status && spctl --status && fdesetup status`

Функція: перевірка трьох механізмів безпеки macOS: `csrutil status` — статус SIP (System Integrity Protection); `spctl --status` — статус політики безпеки Gatekeeper; `fdesetup status` — статус повного шифрування диска FileVault.

13. `codesign --verify --verbose /Applications/Safari.app && ls -lO /System`

Функція: `codesign --verify --verbose` перевіряє цілісність цифрового підпису програми Apple; `ls -lO` з прапорцем `-O` виводить системні прапорці файлів (показує прапорець `restricted`, який захищає каталоги навіть від root).

14. `[FreeBSD] freebsd-version -kru`

Функція: виводить версію FreeBSD. **Параметри:** `-k` (встановлене ядро), `-r` (запущене ядро), `-u` (користувацьке середовище userland). Усі три = `15.1-RELEASE`.

15. `[FreeBSD] kldstat && kldload if_bridge`

Функція: робота з модулями монолітного ядра (Kernel Linker). `kldstat` виводить таблицю завантажених модулів (`kernel`, `zfs.ko`); `kldload if_bridge` динамічно підвантажує модуль мережевого моста `if_bridge.ko` та `bridgestp.ko` без перезавантаження.

16. `[FreeBSD] zpool status zroot && zfs list`

Функція: діагностика ZFS. `zpool status zroot` перевіряє стан пулу (ONLINE, 0 помилок READ/WRITE/CKSUM); `zfs list -o name,used,avail,mountpoint` виводить датасети пулу.

17. [FreeBSD] `zfs snapshot zroot/ROOT/default@lab1 && zfs list -t snapshot`

Функція: створює миттєвий Copy-on-Write знімок кореневої ФС з ім'ям `@lab1`; ключ `-t snapshot` виводить список знімків (розмір `USED = 0B`).

18. [FreeBSD] `jail -c name=lab1 path=/ host.hostname=lab1-jail persist`

Функція: створює ізольований контейнер FreeBSD Jail. **Параметри:** `-c` (create), `name=lab1` (ім'я), `path=/` (корінь), `host.hostname=...` (власне ім'я хоста), `persist` (не видаляти автоматично без процесів).

19. [FreeBSD] `jls && jexec lab1 ps ax && jail -r lab1`

Функція: `jls` (jail list) показує активні jails (JID 3); `jexec lab1 ps ax` запускає `ps ax` всередині jail (бачить лише власний процес з прапорцем `J`); `jail -r lab1` (`-r` remove) видаляє jail.

20. [FreeBSD] `pkg -v && pkg info | wc -l && pkg info fastfetch`

Функція: бінарний пакетний менеджер FreeBSD. `-v` — версія (2.8.4), `pkg info | wc -l` — підрахунок усіх встановлених пакетів (496, включно з базовою системою FreeBSD 15 PkgBase).

📊 ПОЯСНЕННЯ РЕЗУЛЬТАТІВ (ЧОМУ ТАК ВИЙШЛО)

Головний висновок з коду: Усі 20 команд наочно доводять: попри спільні утиліти BSD (`sysctl`, `ps`, `mount`, `df`), macOS має гібридне ядро XNU (`kmutil`), ініціалізатор `launchd` і файлову систему APFS, тоді як FreeBSD має модульне монолітне ядро (`kldstat`), класичний `init`, файлову систему ZFS і контейнерну ізоляцію `jails`.

👉 ІНСТРУКЦІЯ ДЛЯ ТЕБЕ (ЩО НАТИСНУТИ / ПОКАЗАТИ)

Натисни кнопку **Копіювати** біля будь-якої команди macOS і встав у Термінал (Cmd+V), якщо викладач попросить продемонструвати окрему команду.

1 Титульна сторінка, Мета і Вибір ОС

Сторінки 1–2 звіту · Розділ 1

🎙️ ЩО КАЗАТИ ВИКЛАДАЧУ (СЦЕНАРІЙ ДОПОВІДІ)

Добрий день! Я Загорський Тарас, група ІПЗ-31. Представляю лабораторну роботу № 1 «Порівняння операційних систем».

Мета роботи — дослідити архітектурні відмінності операційних систем, визначити їхні функції, переваги та недоліки. Для порівняння зі списку я обрав **macOS (№ 6)** та **FreeBSD (№ 5)**.

Чому саме ця пара: обидві системи мають спільне історичне коріння UNIX та 4.4BSD. Значна частина ядра macOS (шар BSD у XNU), мережевий стек і базові консольні утиліти взяті безпосередньо з FreeBSD. Але далі вони розвивалися у протилежних напрямках: macOS стала закритою комерційною десктопною ОС для комп'ютерів Apple, а FreeBSD — відкритою серверною операційною системою.

📊 ПОЯСНЕННЯ РЕЗУЛЬТАТІВ (ЧОМУ ТАК ВИЙШЛО)

Пояснення для викладача: Вибір цієї пари дозволяє порівняти не просто дві різні ОС, а два еволюційні шляхи однієї гілки BSD: клієнтський (орієнтований на GUI, мультимедіа та енергоефективність ARM) та серверний (орієнтований на мережеву продуктивність, ZFS та аптайм).

👉 ІНСТРУКЦІЯ ДЛЯ ТЕБЕ (ЩО НАТИСНУТИ / ПОКАЗАТИ)

Що показати: Відкрий PDF-звіт на сторінках 1–2, покажи розділ «1. Вибір операційних систем».

3 Загальна характеристика і призначення

Сторінки 3–4 звіту · Таблиця 1 · Розділ 3–4

🎙️ ЩО КАЗАТИ ВИКЛАДАЧУ (СЦЕНАРІЙ ДОПОВІДІ)

У таблиці 1 наведено порівняльну характеристику. macOS вийшла у 2001 році на базі NeXTSTEP, має **гібридне ядро XNU** (мікроядро Mach + шар BSD + об'єктно-орієнтована підсистема драйверів IOKit), пропрієтарну ліцензію з відкритим ядром Darwin і офіційну сертифікацію **UNIX 03**.

FreeBSD з'явилася у 1993 році, має **монолітне ядро з динамічно завантажуваними модулями (kld)**, вільну двопунктову BSD-ліцензію і підтримує багато архітектур (amd64, arm64, riscv64). За призначенням macOS — це робочі станції та ноутбуки Apple, а FreeBSD — високонавантажені сервери, системи зберігання даних і маршрутизатори (на ній працюють сервери CDN Netflix, ОС консолей PlayStation 4/5 та брандмауер pfSense).

📊 ПОЯСНЕННЯ РЕЗУЛЬТАТІВ (ЧОМУ ТАК ВИЙШЛО)

Пояснення результатів Таблиці 1: Сертифікація UNIX 03 у macOS означає повну відповідність стандарту Single UNIX Specification v3, а 2-clause BSD ліцензія у FreeBSD дозволяє корпораціям (Apple, Sony, Netflix, Juniper) брати код FreeBSD у закриті продукти без вимоги відкривати власні напрацювання (на відміну від GPL у Linux).

👉 ІНСТРУКЦІЯ ДЛЯ ТЕБЕ (ЩО НАТИСНУТИ / ПОКАЗАТИ)

Що показати: Таблицю 1 на сторінці 3 звіту (рядки «Архітектура ядра», «Ліцензія», «Файлові системи»).

5 Практика macOS: Система, ядро XNU та Пам'ять

Сторінки 5–6 звіту · Рисунки 1–3

🎙 ЩО КАЗАТИ ВИКЛАДАЧУ (СЦЕНАРІЙ ДОПОВІДІ)

Щоб перевірити теорію на практиці, я дослідив обидві ОС вживу. Спочатку macOS 27.2 на своєму MacBook Pro з процесором M1 Pro.

На рисунку 1 утиліти `sw_vers`, `uname -a` та `sysctl` показують ядро Darwin 27.2.0 (гілка XNU 13432) для архітектури ARM64, 10 ядер процесора і 16 ГБ оперативної пам'яті. На рисунку 2 команда `kmutil showloaded` демонструє три ключові інтерфейси ядра: `com.apple.kpi.mach`, `com.apple.kpi.bsd` та `com.apple.kpi.iokit` — це практичний доказ гібридної архітектури XNU. А команда `ps` показує, що першим процесом системи з PID 1 є `launchd`.

На рисунку 3 у програмі «Моніторинг системи» видно механізм стисненої пам'яті (Compressed Memory): замість того, щоб одразу скидати неактивні сторінки у swap на SSD, ядро XNU стискає їх у швидкій оперативній пам'яті.

💻 ПОЯСНЕННЯ КОДУ, ФУНКЦІЙ ТА ПАРАМЕТРІВ

```
sw_vers && uname -a
```

Виводить версію macOS 27.2 (26B123) та ядро Darwin 27.2.0 XNU arm64.

```
sysctl kern.ostype kern.osrelease hw.ncpu hw.memsize
```

Виводить тип ядра Darwin, 10 ядер CPU та 17179869184 байт (16 ГБ) ОЗП.

```
ps -axco pid,user,%cpu,%mem,comm | head -12
```

Показує процес PID 1 — launchd (єдиний менеджер служб і демонів macOS).

```
kmutil showloaded --list-only 2>/dev/null | awk '{print $1, $6, $7}' | head -10
```

Виводить завантажені компоненти ядра: kpi.bsd, kpi.iokit, kpi.mach.

📊 ПОЯСНЕННЯ РЕЗУЛЬТАТІВ (ЧОМУ ТАК ВИЙШЛО)

Пояснення результатів (Рис. 1–3): У гібридному ядрі XNU мікроядро **Mach** відповідає за планувальник потоків, віртуальну пам'ять та міжпроцесну взаємодію (IPC); шар **BSD** — за POSIX API, файлову систему, мережу і права доступу; **IOKit** — за драйвери пристроїв. Усі три частини працюють в одному адресному просторі ядра заради швидкодії.

👉 **ІНСТРУКЦІЯ ДЛЯ ТЕБЕ (ЩО НАТИСНУТИ / ПОКАЗАТИ)**

Що показати: Рисунок 2 у звіті (рядки `kpi.bsd`, `kpi.iokit`, `kpi.mach`) або відкрите вікно Термінала та «Моніторинг системи».

7 Практика macOS: Файлова система APFS та Безпека

Сторінки 7–8 звіту · Рисунки 4–7

🎙 ЩО КАЗАТИ ВИКЛАДАЧУ (СЦЕНАРІЙ ДОПОВІДІ)

Далі я дослідив файлову систему APFS (рисунки 4–5). Команда `diskutil list internal` показує, що фізичний SSD містить контейнер APFS `disk3` на 494 ГБ, у якому живуть 6 логічних томів: системний `Macintosh HD`, том користувацьких даних `Data`, `Preboot`, `Recovery` та `VM`. Вони ділять спільний пул вільного простору (Space Sharing).

Команда `mount` показує, що кореневий том `/` змонтований не напряму з диска, а з криптографічно підписаного незмінного знімка APFS з прапорцями `sealed, read-only`.

На рисунках 6–7 перевірено багаторівневу безпеку macOS: `csrutil status` підтверджує увімкнений захист цілісності системи SIP, `spctl --status` — роботу Gatekeeper, `fdesetup status` — апаратне шифрування диска FileVault, а `ls -lO /System` показує прапорець `restricted`, який забороняє модифікацію системних каталогів навіть суперкористувачу root.

💻 ПОЯСНЕННЯ КОДУ, ФУНКЦІЙ ТА ПАРАМЕТРІВ

```
diskutil list internal | head -22
```

Виводить структуру контейнера APFS disk3 та всі логічні томи.

```
mount | grep -E 'apfs' | head -5
```

Показує монтування кореня / як apfs, sealed, local, read-only, journaled.

```
df -h / /System/Volumes/Data
```

Показує однаковий доступний простір (Avail 82Gi) для системного тому і тому даних.

```
csrutil status && spctl --status && fdesetup status && ls -lO /System | hea
d -6
```

Перевіряє SIP, Gatekeeper, FileVault і прапорець restricted.

📊 ПОЯСНЕННЯ РЕЗУЛЬТАТІВ (ЧОМУ ТАК ВИЙШЛО)

Пояснення результатів (Рис. 4–7): Концепція **Signed System Volume (SSV)** у macOS гарантує, що навіть якщо зловмисник отримає права root, він не зможе змінити системні файли, бо корінь змонтований у режимі `read-only` із

запечатаного знімка, хеш-дерево якого перевіряється чіпом Apple Silicon під час завантаження.

👉 **ІНСТРУКЦІЯ ДЛЯ ТЕБЕ (ЩО НАТИСНУТИ / ПОКАЗАТИ)**

Що показати: На рис. 4 рядок `/dev/disk3s1s1 on / (apfs, sealed, local, read-only)`, а на рис. 6 — прапорець `restricted`.

9 Практика FreeBSD: Система і модульне ядро

Сторінки 9–10 звіту · Рисунки 8–10

🎙️ ЩО КАЗАТИ ВИКЛАДАЧУ (СЦЕНАРІЙ ДОПОВІДІ)

Другу частину дослідження я провів у FreeBSD 15.1-RELEASE для arm64 у віртуальній машині QEMU (рисунки 8–9). Команда `freebsd-version -kru` вивела три однакових рядки `15.1-RELEASE` — для встановленого ядра, запущеного ядра та призначеного для користувача оточення (userland). Це ключова відмінність FreeBSD від Linux: тут ядро і всі базові утиліти розробляються однією командою як єдина цілісна система.

На рисунку 10 я перевірив модульність монолітного ядра. Спочатку `kldstat` показав лише ядро `kernel` і модуль файлової системи `zfs.ko`. Потім командою `kldload if_bridge` я прямо під час роботи системи, без перезавантаження, підвантажив модуль мережевого моста — і у виводі `kldstat` одразу з'явилися `if_bridge.ko` та `bridgestp.ko`. А команда `ps` показала, що першим процесом (PID 1) тут є класичний `init`, а PID 0 — це саме ядро (`kernel`).

💻 ПОЯСНЕННЯ КОДУ, ФУНКЦІЙ ТА ПАРАМЕТРІВ

```
freebsd-version -kru && uname -a
```

Перевіряє синхронність версій ядра (-k, -r) і базової системи (-u) = 15.1-RELEASE GENERIC arm64.

```
kldstat && kldload if_bridge && kldstat
```

Показує динамічне підвантаження модуля ядра if_bridge.ko на льоту.

```
ps -ax -o pid,user,%cpu,%mem,comm | head -10
```

Виводить процеси ядра (PID 0 kernel, zfskern, pagedaemon) та PID 1 init.

📊 ПОЯСНЕННЯ РЕЗУЛЬТАТІВ (ЧОМУ ТАК ВИЙШЛО)

Пояснення результатів (Рис. 8–10): Хоча ядро FreeBSD монолітне (всі драйвери працюють у спільному адресному просторі ядра), механізм **KLD (Kernel Linker)** дозволяє не тримати всі драйвери в пам'яті постійно, а підключати їх динамічно як файли `.ko`.

👉 ІНСТРУКЦІЯ ДЛЯ ТЕБЕ (ЩО НАТИСНУТИ / ПОКАЗАТИ)

Що показати: Рисунок 10 у звіті — порівняй перший і другий вивід `kldstat` та вкажи на `PID 1 init`.

11 Практика FreeBSD: ZFS, Jails та пакети pkg

Сторінки 11–12 звіту · Рисунки 11–13 · Таблиця 2

🎙 ЩО КАЗАТИ ВИКЛАДАЧУ (СЦЕНАРІЙ ДОПОВІДІ)

На рисунку 11 досліджено файлову систему **ZFS**. Команда `zpool status zroot` показує стан ONLINE і нуль помилок контрольних сум (CKSUM). Командою `zfs snapshot zroot/ROOT/default@lab1` я за долю секунди створив знімок усієї кореневої файлової системи на 2,59 ГБ. У списку знімків видно, що `USED = 0B`, оскільки ZFS використовує принцип Copy-on-Write і не копіює блоки даних, поки вони не зміняться.

На рисунку 12 показано фірмовий механізм ізоляції **FreeBSD Jails**. Командою `jail -c` я створив ізольоване середовище `lab1` з власним hostname `lab1-jail`. Коли я запустив всередині нього `jexec lab1 ps ax`, процес побачив лише самого себе — усі процеси хоста для нього приховані.

На рисунку 13 показано пакетний менеджер `pkg`: у FreeBSD 15 навіть базова система керується пакетами (PkgBase), тому встановлено 496 пакетів. Таблиця 2 підсумовує всі практичні відмінності.

💻 ПОЯСНЕННЯ КОДУ, ФУНКЦІЙ ТА ПАРАМЕТРІВ

```
zpool status zroot && zfs list -o name,used,avail,mountpoint | head -8
```

Перевіряє здоров'я пулу ZFS та ієрархію наборів даних.

```
zfs snapshot zroot/ROOT/default@lab1 && zfs list -t snapshot
```

Створює миттєвий знімок ФС, що займає 0 байт (USED 0B, REFER 2.59G).

```
jail -c name=lab1 path=/ host.hostname=lab1-jail persist && jls && jexec lab1 ps ax && jail -r lab1
```

Створює контейнер Jail, перевіряє ізоляцію процесів і видаляє його.

📊 ПОЯСНЕННЯ РЕЗУЛЬТАТІВ (ЧОМУ ТАК ВИЙШЛО)

Пояснення результатів (Рис. 11–13): ZFS об'єднує функції файлової системи і менеджера логічних томів, автоматично перевіряючи контрольні суми кожного блока при читанні (захист від тихого пошкодження даних bit rot). А Jails стали прообразом сучасних Docker-контейнерів ще у 2000 році.

👉 ІНСТРУКЦІЯ ДЛЯ ТЕБЕ (ЩО НАТИСНУТИ / ПОКАЗАТИ)

Що показати: На рис. 11 рядок `USED 0B`, на рис. 12 — вивід `jexec lab1 ps ax` (де видно лише 1 процес з літерою `J` у стовпці STAT).

13 Переваги, недоліки та підсумкове порівняння

Сторінки 13–15 звіту · Розділи 5–7 · Таблиця 3

🎙️ **ЩО КАЗАТИ ВИКЛАДАЧУ (СЦЕНАРІЙ ДОПОВІДІ)**

У розділах 5 і 6 я виділив по 6 переваг та 6 недоліків кожної ОС.

Переваги macOS: тісна інтеграція власного заліза Apple Silicon та ОС (висока автономність і стабільність), зручний графічний інтерфейс Aqua, сильна безпека (SIP, Gatekeeper, FileVault, Secure Enclave), сертифікована UNIX-основа, екосистема з iPhone/iPad та наявність професійного ПЗ. **Недоліки macOS:** прив'язка тільки до дорогого обладнання Apple, закритість коду GUI, неможливість апгрейду ОЗП і SSD, мало ігор та відсутність серверної редакції.

Переваги FreeBSD: еталонна стабільність і продуктивний мережевий стек (Netflix віддає сотні Гбіт/с з одного сервера), цілісність ядра і userland, вбудовані ZFS та Jails, вільна BSD-ліцензія і безкоштовність. **Недоліки FreeBSD:** менше драйверів для ноутбуків, Wi-Fi та відеокарт, відсутність графіки за замовчуванням, відсутність нативної підтримки Docker/Kubernetes.

📊 **ПОЯСНЕННЯ РЕЗУЛЬТАТІВ (ЧОМУ ТАК ВИЙШЛО)**

Пояснення Таблиці 3 (стор. 15): У таблиці 3 видно, що macOS перемагає у зручності для користувача, мультимедіа та дизайні, а FreeBSD — у серверному застосуванні, відкритості коду, файловій системі ZFS та вбудованій віртуалізації (Jails + bhyve). За рівнем безпеки обидві системи мають паритет.

👉 **ІНСТРУКЦІЯ ДЛЯ ТЕБЕ (ЩО НАТИСНУТИ / ПОКАЗАТИ)**

Що показати: Таблицю 3 на сторінці 15 (останній стовпець «Перевага»).

19 Висновки та Контрольні питання (ЛР 1)

Сторінки 16–20 звіту · Відповіді на питання

ЩО КАЗАТИ ВИКЛАДАЧУ (СЦЕНАРІЙ ДОПОВІДІ)

Висновки: Ми на практиці переконалися, що macOS і FreeBSD, маючи спільне коріння BSD, еволюціонували під абсолютно різні задачі. macOS — це еталонна захищена настільна система з гібридним ядром XNU та APFS для кінцевого користувача і розробника. FreeBSD — це надійна відкрита серверна платформа з модульним монолітним ядром, ZFS та Jails. Отже, універсальної «найкращої» ОС не існує — вибір завжди визначається задачею.

1. Дайте визначення поняття «операційна система». +

2. Назвіть приклади прикладного та системного ПЗ. +

3. Дайте визначення понять «API», «драйвер», «ядро». +

4. Наведіть способи класифікації ОС. +

5. Вимоги до сучасних ОС. +

6. Основні функції ОС. +

Підготовка на Mac та Контекст роботи (ЛР 2)

Все, що вже налаштовано на твоєму Mac і як це запустити

🚀 ШПАРГАЛКА ПІДГОТОВКИ ТА КОНТЕКСТ (ДЛЯ ТЕБЕ НА MAC)

Контекст того, як зроблена робота: У тебе Mac на процесорі **Apple M1 Pro (ARM64)**. На ньому встановлено **Oracle VirtualBox 7.2.22** для Apple Silicon. Оскільки VirtualBox на M1 використовує апаратну віртуалізацію `Hypervisor.framework` без емуляції x86, 64-бітна Windows 10 x86 на ньому фізично не стартує — тому встановлена офіційна **Windows 11 Pro ARM64 (збірка 26300.9457)** у ВМ `Zahorskyi_Win`.

- **1. Автоматичний запуск всього одною командою в Терміналі (macOS):**
  `cd "/Users/taras/Desktop/unik/Operation Systme" && ./start_defense.sh 2`
  Скрипт сам відкриє PDF-звіт ЛР2, відкриє у Finder папку ВМ `~/VirtualBox VMs/Zahorskyi_Win`*, спільну теку* `lab2/Спільна_тека`*, папку* `Export` *з готовим* `Zahorskyi_Win.ova` *(15 ГБ), виведе всі параметри* `VBoxManage` *у термінал і запустить саму ВМ* `Zahorskyi_Win`*!*
- **2. Логін та пароль у Windows 11 (якщо спитає):**
  - Користувач: `taras` (або `Taras`) — входить автоматично (AutoLogon).
  - Пароль (якщо заблокується екран): `Lab2pass` або `lab`.
- **3. Де лежать усі файли на твоєму Mac (щоб показати у Finder):**
  - Папка ВМ: `/Users/taras/VirtualBox VMs/Zahorskyi_Win/` (тут лежать `Zahorskyi_Win.vbox`, `.vbox-prev`, `Zahorskyi_Win.vdi` на 14,9 ГБ, `USB_Flash.vdi`, `.nvram`, папки `Logs` і `Snapshots`).
  - Онлайн-знімок у `Snapshots`: `Lab2_snapshot` (два диференціальні диски `{UUID}.vdi` по 2 МБ + дамп пам'яті `.sav` на 2,1 ГБ).
  - Експортований OVA-архів: `/Users/taras/VirtualBox VMs/Export/Zahorskyi_Win.ova` (15 ГБ).
  - Спільна тека на Mac: `/Users/taras/Desktop/unik/Operation Systme/lab2/Спільна_тека` (у Windows це мережевий диск `Z:`).

💻 ПОЯСНЕННЯ КОДУ, ФУНКЦІЙ ТА ПАРАМЕТРІВ

```
cd "/Users/taras/Desktop/unik/Operation Systme" && ./start_defense.sh 2
```

Запускає повну підготовку ЛР2: відкриває PDF, папки ВМ у Finder, виводить VBoxManage інфо і стартує Zahorskyi_Win.

20 Построковий розбір усіх 20 команд звіту (Шпаргалка)

Команди VBoxManage, PowerShell та macOS зі звіту ЛР 2

🎙 ЩО КАЗАТИ ВИКЛАДАЧУ (СЦЕНАРІЙ ДОПОВІДІ)

У ході виконання лабораторної роботи № 2 я використовував консольний інтерфейс керування гіпервізором `VBoxManage` на хості macOS та командлети PowerShell/cmd всередині гостьової Windows 11. Ось повний построковий розбір усіх 20 команд звіту з їхніми параметрами.

💻 ПОЯСНЕННЯ КОДУ, ФУНКЦІЙ ТА ПАРАМЕТРІВ

```
1. brew install --cask virtualbox && VBoxManage --version
```

Функція: встановлює пакет VirtualBox для macOS ARM64 через менеджер Homebrew (`--cask` — для графічних macOS-програм) і виводить версію гіпервізора (`7.2.22r175245`).

```
2. VBoxManage list ostypes | grep -B1 -A3 'Windows11_arm64'
```

Функція: виводить підтримувані профілі гостьових ОС. Доводить наявність профілю `Windows11_arm64` (Windows 11 on ARM 64-bit).

```
3. VBoxManage createvm --name Zahorskyi_Win --ostype Windows11_arm64 --regi
ster
```

Функція: створює нову ВМ та XML-файл `Zahorskyi_Win.vbox` . **Параметри:** `--name` (ім'я ВМ і папки), `--ostype` (архітектура ARM64), `--register` (реєструє ВМ у глобальному `VirtualBox.xml`).

```
4. VBoxManage modifyvm Zahorskyi_Win --memory 4096 --cpus 4 --vram 128 --fi
rmware efi
```

Функція: задає апаратні ресурси ВМ. **Параметри:** `--memory 4096` (4 ГБ ОЗП), `--cpus 4` (4 ядра), `--vram 128` (128 МБ відеопам'яті), `--firmware efi` (прошивка UEFI + TPM 2.0 + Secure Boot для Win 11).

```
5. VBoxManage modifyvm Zahorskyi_Win --clipboard-mode bidirectional --drag-
and-drop bidirectional
```

Функція: вмикає двосторонній спільний буфер обміну та режим перетягування об'єктів між хостом і гостем (`bidirectional`).

```
6. VBoxManage modifyvm Zahorskyi_Win --boot1 disk --boot2 dvd --boot3 none
--boot4 none
```

Функція: встановлює порядок завантаження у прошивці: 1-й пристрій — жорсткий диск (`disk`), 2-й — оптичний привід (`dvd`).

7. `VBoxManage modifyvm Zahorskyi_Win --nic1 nat --nictype1 virtio --cableconnected1 on --usb-xhci on`

Функція: налаштовує мережу та USB. **Параметри:** `--nic1 nat` (режим трансляції мережевих адрес NAT), `--nictype1 virtio` (паравіртуальна мережева карта virtio-net), `--usb-xhci on` (контролер USB 3.0 xHCI).

8. `VBoxManage createmedium disk --filename Zahorskyi_Win.vdi --size 65536 --format VDI --variant Standard`

Функція: створює віртуальний жорсткий диск. **Параметри:** `--size 65536` (максимальна ємність 64 ГБ у МБ), `--format VDI` (рідний формат VirtualBox), `--variant Standard` (динамічно розширюваний диск).

9. `VBoxManage storageattach Zahorskyi_Win --storagectl SATA --port 0 --type hdd --medium Zahorskyi_Win.vdi`

Функція: підключає створений VDI-диск до порту 0 контролера SATA AHCI (`--type hdd`), а на порт 1 (`--type dvddrive`) — ISO-образ Windows 11.

10. `VBoxManage storagectl Zahorskyi_Win --name USB --add usb && VBoxManage storageattach ... USB_Flash.vdi`

Функція: додає контролер носіїв на шині USB (`--add usb`) і приєднує до нього образ `USB_Flash.vdi` (1 ГБ) з прапорцем `--hotpluggable on` , який Windows бачить як справжню USB-флешку.

11. `VBoxManage sharedfolder add Zahorskyi_Win --name 'Спільна_тека' --hostpath '.../lab2/Спільна_тека' --automount --auto-mount-point Z:`

Функція: прокидає папку macOS у гостьову Windows. **Параметри:** `--automount` (автопідключення при старті), `--auto-mount-point Z:` (монтувати як мережевий диск Z:).

12. `VBoxManage unattended install Zahorskyi_Win --iso=Windows11...iso --user=taras --install-additions --start-vm=gui`

Функція: автоматичне встановлення Windows 11. Генерує `autounattend.xml` , ставить систему, монтує `VBoxGuestAdditions.iso` (`--install-additions`) і запускає ВМ.

13. `VBoxManage showvminfo Zahorskyi_Win --machinereadable`

Функція: виводить повну поточну конфігурацію ВМ (рис. 5) та стан служб Guest Additions (рис. 6: `VBoxService.exe 7.2.22`).

14. `[macOS ⟷ Win] pbcopy / pbpaste та Get-Clipboard / Set-Clipboard`

Функція: перевірка двонаправленого буфера обміну (рис. 7–8). `pbpaste` на macOS миттєво виводить текст, скопійований у Блокноті Windows.

15. `[Win PowerShell] pnputil /add-driver Z:\netkvm_arm64\*.inf /install`

Функція: встановлює у Windows ARM64 підписаний драйвер паравіртуального мережевого адаптера Red Hat VirtIO (`NetKVM`) зі спільної теки Z:.

```
16. [Win PowerShell] Get-NetIPConfiguration; ping -n 3 google.com; Invoke-WebRequest https://www.microsoft.com -UseBasicParsing
```

Функція: перевірка мережі NAT (рис. 9). Показує отриману по DHCP адресу `10.0.2.15` , шлюз `10.0.2.2` , 0% втрат ping (`-n 3` — 3 пакети) і HTTP StatusCode `200 OK` .

```
17. [Win PowerShell] Get-Disk; Get-Volume; dir \\VBoxSvr\Спільна_тека
```

Функція: перевірка носіїв (рис. 10–11). `Get-Disk` показує `VBOX HARDDISK` на шині **USB** (том E: USB_FLASH FAT32), `Get-Volume` показує CD-ROM (D:), а `dir \\VBoxSvr\...` — файли у спільній теці Z:.

```
18. VBoxManage showmediuminfo disk Zahorskyi_Win.vdi && ls -lh '~/VirtualBox VMs/Zahorskyi_Win'
```

Функція: дослідження файлів ВМ (рис. 12). Доводить економію місця динамічного диска: логічна ємність `Capacity: 65536 MBytes` (64 ГБ), а фактичний розмір на SSD мака `Size on disk: 14899 MBytes` (14,9 ГБ).

```
19. VBoxManage snapshot Zahorskyi_Win take 'Lab2_snapshot' --live && VBoxManage controlvm Zahorskyi_Win savestate
```

Функція: `snapshot ... take --live` робить онлайновий знімок без вимкнення ВМ (створює диференціальні `{UUID}.vdi` та `.sav` у папці `Snapshots` , рис. 13); `controlvm ... savestate` зберігає стан ВМ у файл `.sav` і присипляє її (рис. 14).

```
20. VBoxManage export Zahorskyi_Win -o Export/Zahorskyi_Win.ova --ovf20 --manifest && tar -tvf Export/Zahorskyi_Win.ova
```

Функція: експортує вимкнену ВМ у стандарт OVF 2.0 (`--ovf20`) зі створенням файлу контрольних сум SHA (`--manifest`). `tar -tvf` показує вміст OVA-архіву: `.ovf` , стиснутий диск `.vmdk` , `.nvram` та `.mf` (рис. 15).

📊 ПОЯСНЕННЯ РЕЗУЛЬТАТІВ (ЧОМУ ТАК ВИЙШЛО)

Підсумок по коду: Усі операції VirtualBox мають подвійний інтерфейс: графічний (VirtualBox Manager) і консольний (`VBoxManage`), які працюють з одним і тим самим XML-файлом `Zahorskyi_Win.vbox` .

👉 ІНСТРУКЦІЯ ДЛЯ ТЕБЕ (ЩО НАТИСНУТИ / ПОКАЗАТИ)

Використовуй цю шпаргалку, коли викладач питає: «А якою командою ти зробив знімок / примонтував диск / перевірив мережу?».

1 Мета, встановлення VirtualBox та особливість ARM64

Сторінки 1–3 звіту · Рисунок 1

🎙️ ЩО КАЗАТИ ВИКЛАДАЧУ (СЦЕНАРІЙ ДОПОВІДІ)

Добрий день! Представляю лабораторну роботу № 2 «Встановлення і використання Oracle VM VirtualBox». Основна ОС — **macOS 27.2** на MacBook Pro з процесором Apple M1 Pro, гостьова ОС — **Windows 11 Pro**.

VirtualBox 7.2.22 я встановив через пакетний менеджер Homebrew. Важливий технічний нюанс: на процесорах Apple Silicon VirtualBox працює як гіпервізор типу 2 поверх системного `Hypervisor.framework` і не має програмного емулятора x86. Тобто він запускає виключно ARM64-системи. Оскільки Windows 10 для ARM64 офіційно вже не розповсюджується, я використав офіційний ISO-образ **Windows 11 Pro ARM64** з сайту Microsoft і профіль `Windows 11 on ARM (64-bit)` (рисунок 1).

📊 ПОЯСНЕННЯ РЕЗУЛЬТАТІВ (ЧОМУ ТАК ВИЙШЛО)

Пояснення результатів (Рис. 1): Використання апаратної віртуалізації ARM-на-ARM забезпечує майже нативну швидкодію процесора, оскільки інструкції гостьової ОС виконуються безпосередньо на фізичних ядрах Apple M1 Pro без бінарної трансляції.

👉 ІНСТРУКЦІЯ ДЛЯ ТЕБЕ (ЩО НАТИСНУТИ / ПОКАЗАТИ)

Що показати: Рисунок 1 у звіті — рядок `ID: Windows11_arm64` та версію `7.2.22r175245`.

4 Створення ВМ Zahorskyi_Win та встановлення Windows 11

Сторінки 4–6 звіту · Рисунки 2–4

🎙️ ЩО КАЗАТИ ВИКЛАДАЧУ (СЦЕНАРІЙ ДОПОВІДІ)

Я створив віртуальну машину з назвою **Zahorskyi_Win** (рисунок 2). Виділив їй 4 ГБ оперативної пам'яті (з 16 ГБ доступних на Mac), 4 віртуальні процесори, 128 МБ відеопам'яті та динамічний віртуальний диск формату VDI максимальним обсягом 64 ГБ. Оскільки це Windows 11, увімкнено мікропрограму UEFI, Secure Boot та віртуальний криптомодуль TPM 2.0.

Для встановлення ОС я використав режим **Unattended Install** (автоматичне встановлення): VirtualBox сам згенерував файл відповідей `autounattend.xml` з обліковим записом `taras`, українською локаллю та часовим поясом і встановив систему без ручного вводу (рисунки 3–4).

📊 ПОЯСНЕННЯ РЕЗУЛЬТАТІВ (ЧОМУ ТАК ВИЙШЛО)

Пояснення результатів (Рис. 2–4): У вікні гостьової Windows 11 команда `winver` та `systeminfo` підтверджують встановлення Windows 11 Pro 26H2 (збірка 26300.9457) на віртуальній платформі VirtualBox з 4 процесорами Apple Silicon і 4096 МБ ОЗП.

👉 ІНСТРУКЦІЯ ДЛЯ ТЕБЕ (ЩО НАТИСНУТИ / ПОКАЗАТИ)

Що показати: На рис. 2 параметри ВМ у менеджері VirtualBox, на рис. 4 — вікно запущеної Windows 11.

7 Налаштування ВМ та Гостьові доповнення (Guest Additions)

Сторінки 7–8 звіту · Рисунки 5–6

🎙️ ЩО КАЗАТИ ВИКЛАДАЧУ (СЦЕНАРІЙ ДОПОВІДІ)

У розділі 4 я виконав усі обов'язкові налаштування за завданням і перевірив їх командою `VBoxManage showvminfo` (рисунок 5): двонаправлений буфер обміну і drag'n'drop (`bidirectional`), порядок завантаження — спочатку жорсткий диск (`boot1=disk`), потім оптичний привід (`boot2=dvd`), підключений ISO-образ у CD/DVD, мережа в режимі `NAT`, контролер USB 3.0 (`xHCI`) і спільна тека з автопідключенням на диск `Z:`.

Потім встановлено **VirtualBox Guest Additions 7.2.22** з образу `VBoxGuestAdditions.iso` (рисунок 6). У системі з'явилися драйвери `VBoxGuest.sys`, `VBoxSF.sys` (спільні теки), `VBoxMouse.sys` і фонова служба `VBoxService.exe`, яка передає гіпервізору властивості гостя (версію ОС, IP, логін).

📊 ПОЯСНЕННЯ РЕЗУЛЬТАТІВ (ЧОМУ ТАК ВИЙШЛО)

Пояснення результатів (Рис. 5–6): Без гостьових доповнень віртуальна машина повністю ізольована. Саме драйвери Guest Additions відкривають канал зв'язку через віртуальний PCI-пристрій VMMDev між гостьовою ОС і хостом для спільного буфера, спільних тек та інтеграції курсора миші.

👉 ІНСТРУКЦІЯ ДЛЯ ТЕБЕ (ЩО НАТИСНУТИ / ПОКАЗАТИ)

Що показати: На рис. 5 рядки `clipboard="bidirectional"`, `boot1="disk"`, `boot2="dvd"`; на рис. 6 — службу `VBoxService Running`.

9 Перевірка буфера обміну, Drag'n'Drop та вирішення проблеми з мережею NAT

Сторінки 9–10 звіту · Рисунки 7–9

🎙️ ЩО КАЗАТИ ВИКЛАДАЧУ (СЦЕНАРІЙ ДОПОВІДІ)

На рисунках 7 і 8 перевірено двонаправлений буфер обміну: український текст, скопійований у macOS, вставився Ctrl+V у Блокнот Windows без спотворень кодування, а рядок з Windows миттєво зчитався на Mac командою `pbpaste`. Натомість **drag'n'drop**, хоч і увімкнений у налаштуваннях, не працює: у документації Oracle вказано, що на хостах macOS перетягування файлів між Finder і гостьовою ОС не реалізоване.

При налаштуванні мережі NAT (рисунок 9) виникла цікава інженерна проблема: стандартний для ARM адаптер «Ethernet over USB» у Windows 11 видавав помилку запуску (Код 10), а для карт Intel PRO/1000 у Windows ARM64 немає драйверів. Я вирішив це так: перемкнув тип адаптера на паравіртуальний `virtio-net` і через спільну теку встановив підписаний ARM64-драйвер **Red Hat VirtIO (NetKVM)**. Після цього Windows отримала від DHCP-сервера VirtualBox IP `10.0.2.15`, шлюз `10.0.2.2`, `ping google.com` пройшов з 0% втрат, а `Invoke-WebRequest` до microsoft.com повернув `StatusCode 200`.

📊 ПОЯСНЕННЯ РЕЗУЛЬТАТІВ (ЧОМУ ТАК ВИЙШЛО)

Пояснення результатів (Рис. 7–9): У режимі **NAT** VirtualBox виступає як домашній роутер: внутрішня мережа ВМ має діапазон `10.0.2.0/24`, сама ВМ отримує `10.0.2.15`, віртуальний маршрутизатор гіпервізора має адресу `10.0.2.2`, а DNS-проксі — `10.0.2.3`.

👉 ІНСТРУКЦІЯ ДЛЯ ТЕБЕ (ЩО НАТИСНУТИ / ПОКАЗАТИ)

Що показати: На рис. 9 — `Red Hat VirtIO Ethernet Adapter`, IP `10.0.2.15` та `Lost = 0 (0% loss)`.

11 USB-накопичувач, CD/DVD, Спільні теки та Таблиця 1

Сторінки 11–13 звіту · Рисунки 10–11 · Таблиця 1

🎙 ЩО КАЗАТИ ВИКЛАДАЧУ (СЦЕНАРІЙ ДОПОВІДІ)

На рисунку 10 показано роботу з USB та CD/DVD. Оскільки фізичної флешки не було, я створив образ `USB_Flash.vdi` на 1 ГБ і підключив його до віртуального USB-контролера. Для гостьової Windows це справжній фізичний USB-пристрій: у виводі `Get-Disk` для диска №1 стоїть `BusType = USB`, спрацювало автовідтворення знімних дисків, том відформатовано у FAT32 з міткою `USB_FLASH (E:)` і записано текстовий файл. Поруч видно том `D:` типу `CD-ROM` з інсталяційним образом Windows.

На рисунку 11 перевірено спільні теки: папка macOS змонтована через драйвер `VBoxSF` як мережевий диск `Z: (\VBoxSvr\Спільна_тека)`. Обмін файлами `from_macOS.txt` та `from_Windows.txt` працює миттєво в обидва боки. Усі результати зведено в Таблицю 1.

📊 ПОЯСНЕННЯ РЕЗУЛЬТАТІВ (ЧОМУ ТАК ВИЙШЛО)

Пояснення результатів (Рис. 10–11, Таблиця 1): Підключення VDI-образу через контролер `--add usb` емулює стандарт USB Mass Storage Bulk-Only Transport (BOT), тому Windows завантажує драйвер `USBSTOR.SYS` і працює з образом достоту як зі звичайною флешкою.

👉 ІНСТРУКЦІЯ ДЛЯ ТЕБЕ (ЩО НАТИСНУТИ / ПОКАЗАТИ)

Що показати: На рис. 10 рядок `1 VBOX HARDDISK ... USB 1024 MB`, на рис. 11 — Провідник Windows з диском `Z:` і вікно Finder поруч.

14 Дослідження файлів ВМ, Знімка (Snapshot), Збереження стану та Експорту (.ova)

Сторінки 14–16 звіту · Рисунки 12–15

🎙 ЩО КАЗАТИ ВИКЛАДАЧУ (СЦЕНАРІЙ ДОПОВІДІ)

У розділі 7 я дослідив внутрішню файлову структуру ВМ у каталозі `~/VirtualBox VMs/Zahorskyi_Win` (рисунок 12):
• `Zahorskyi_Win.vbox` — XML-конфігурація машини, та `.vbox-prev` — її резервна копія;
• `Zahorskyi_Win.vdi` — динамічний диск: команда `showmediuminfo` показує, що з максимальних 64 ГБ на SSD хоста він реально займає лише **14,9 ГБ**;
• `Zahorskyi_Win.nvram` — енергонезалежна пам'ять UEFI та змінні Secure Boot;
• папка `Logs` — журнали роботи гіпервізора `VBox.log`.

На рисунку 13 я зробив онлайновий знімок `Lab2_snapshot`. У папці `Snapshots` з'явилися два диференціальні диски `{UUID}.vdi` по 2 МБ (основний диск тепер заморожений, а всі нові сектори пишуться у дельта-файл) та файл `.sav` на 2,1 ГБ зі стиснутим дампом оперативної пам'яті.

На рисунку 14 я виконав «Зберегти стан машини» (`savestate`): ВМ перейшла у стан `saved` і з'явився ще один файл `.sav`. Як тільки я знову запустив ВМ, вона миттєво відновила роботу з того ж місця, а тимчасовий `.sav` автоматично видалився.

На рисунку 15 я вимкнув ВМ і експортував її у стандарт OVF 2.0 — отримав єдиний архів `Zahorskyi_Win.ova` (15 ГБ), всередині якого лежать XML-опис `.ovf`, стиснутий образ диска `.vmdk`, `.nvram` і маніфест контрольних сум `.mf`.

📊 ПОЯСНЕННЯ РЕЗУЛЬТАТІВ (ЧОМУ ТАК ВИЙШЛО)

Важлива деталь зі звіту (стор. 16): Перед експортом у формат OVF віртуальну USB-флешку довелося тимчасово відключити, тому що відкритий стандарт OVF підтримує диски на контролерах IDE/SATA/SCSI/VirtIO, але не вміє описувати диски, підключені до шини USB.

👉 ІНСТРУКЦІЯ ДЛЯ ТЕБЕ (ЩО НАТИСНУТИ / ПОКАЗАТИ)

Що показати: На рис. 12 — `Capacity: 65536 MBytes` vs `Size on disk: 14899 MBytes`; на рис. 14 — як після відновлення ВМ другий файл `.sav` зник; на рис. 15 — вивід `tar -tvf Zahorskyi_Win.ova`.

Покрокова інструкція живої демонстрації на Mac (якщо попросить показати ВМ)

Куди натискати і що показувати у запущеній Zahorskyi_Win

🎙️ ЩО КАЗАТИ ВИКЛАДАЧУ (СЦЕНАРІЙ ДОПОВІДІ)

Я можу прямо зараз показати працюючу віртуальну машину Windows 11, спільний буфер, мережу, флешку, спільну теку та всі файли у Finder.

👉 ІНСТРУКЦІЯ ДЛЯ ТЕБЕ (ЩО НАТИСНУТИ / ПОКАЗАТИ)

1. **Запуск:** Якщо ще не запустив, виконай у Терміналі `./start_defense.sh 2` (або у вікні VirtualBox натисни «Запустити» на **Zahorskyi_Win**). Пароль Windows (якщо спитає): `Lab2pass` або `lab`.
2. **Буфер обміну:** Виділи будь-яке слово на Mac (Cmd+C) → клікни у Блокнот у Windows і натисни **Ctrl+V**.
3. **CD/DVD, USB-флешка і Спільна тека:** Відкрий у Windows **Провідник → Цей ПК (This PC)**:
   - Диск **D:** — оптичний привід CD-ROM з образом Windows 11;
   - Диск **E: (USB_FLASH)** — підключений USB-накопичувач;
   - Мережевий диск **Z: (Спільна_тека на VBoxSvr)** — спільна папка з твоїм Mac (`lab2/Спільна_тека`).
4. **Мережа NAT:** Відкрий у Windows термінал (Win+R → `cmd`) і введи `ping google.com`.
5. **Завантаження з CD/DVD:** У меню вікна ВМ натисни «Машина → Перезапуск» і одразу клацай клавішу **Esc** → відкриється меню UEFI → **Boot Manager** → обери **UEFI VBOX CD-ROM**.
6. **Файли ВМ на Mac:** Перемкнись у відкриті вікна Finder: покажи `~/VirtualBox VMs/Zahorskyi_Win` (файли `.vbox`, `.vdi`, `Snapshots` з `.sav`) та `~/VirtualBox VMs/Export/Zahorskyi_Win.ova`.

Висновки та Контрольні питання (ЛР 2)

Сторінки 16–20 звіту · 8 питань самоперевірки

ЩО КАЗАТИ ВИКЛАДАЧУ (СЦЕНАРІЙ ДОПОВІДІ)

Висновки: Я опанував повний цикл роботи з VirtualBox 7.2 на macOS Apple Silicon: створив ВМ `Zahorskyi_Win`, встановив Windows 11 ARM64, налаштував інтеграцію через Guest Additions, вирішив проблему з драйвером мережі через паравіртуальний адаптер `virtio-net` і дослідив внутрішню структуру файлів ВМ, диференціальних знімків та OVA-експорту. Дякую, готовий відповісти на запитання!

1. Які переваги та недоліки використання ВМ?

2. Які можливості надають гостьові доповнення ВМ (Guest Additions)?

3. На що впливає об’єм оперативної пам’яті, виділеної для ВМ?

4. В чому полягає різниця між динамічним і фіксованим ВЖД?

5. Що таке знімки ВМ (Snapshots) і де вони знаходяться?

6. Яким чином і для чого здійснюється експорт та імпорт ВМ?

7. Які типи мережевих підключень дозволяє налаштовувати ВМ?

8. Які папки та файли створюються для супроводу ВМ?

Підготовка на Mac та Контекст роботи (ЛР 3)

Все, що вже налаштовано для Ubuntu 24.04 і як запустити

🚀 ШПАРГАЛКА ПІДГОТОВКИ ТА КОНТЕКСТ (ДЛЯ ТЕБЕ НА MAC)

Контекст того, як зроблена робота: У VirtualBox 7.2.22 на твоєму MacBook Pro M1 Pro створена друга віртуальна машина — **Zahorskyi_Ubuntu**, де встановлена офіційна **Ubuntu 24.04.5 LTS Desktop для архітектури ARM64** (ядро Linux 7.0.0-38-generic, 4 ГБ ОЗП, 4 ядра, динамічний диск 25 ГБ, який реально займає 9,3 ГБ).

- **1. Автоматичний запуск всього одною командою в Терміналі (macOS):** `cd "/Users/taras/Desktop/unik/Operation Systme" && ./start_defense.sh 3` *Скрипт сам відкриє PDF-звіт ЛР3, відкриє у Finder папку ВМ* `~/VirtualBox VMs/Zahorskyi_Ubuntu`*, спільну теку* `~/VMs/oslabs/lab3_shared`*, папку* `~/VMs/oslabs/export` *з файлом* `Zahorskyi_Ubuntu.ova` *(4,84 ГБ), виведе конфігурацію ВМ у термінал і відкриє VirtualBox Manager!*
- **2. Як запустити саму ВМ Ubuntu (якщо викладач попросить наживо):**
  - У вікні VirtualBox обери **Zahorskyi_Ubuntu** і натисни «Запустити» (або виконай у терміналі: `VBoxManage startvm "Zahorskyi_Ubuntu" --type gui`).
  - Логін в Ubuntu: `taras`, пароль: `taras` або `lab` (або автоматичний вхід).
  - **Важлива порада для твого Mac (16 ГБ ОЗП):** Не тримай одночасно увімкненими і Windows 11, і Ubuntu — перед запуском Ubuntu закрий Windows (Файл → Закрити → Зберегти стан або Вимкнути), щоб Mac не грівся і не використовував swap!
- **3. Де лежать усі файли на твоєму Mac:**
  - Папка ВМ: `/Users/taras/VirtualBox VMs/Zahorskyi_Ubuntu/` (`.vbox`, `.vbox-prev`, `Zahorskyi_Ubuntu.vdi` на 9,3 ГБ, `USB_flash.vdi`, `.nvram`, папки `Logs` і `Snapshots`).
  - Знімок стану в `Snapshots`: `Після налаштування` (два диференційні `{UUID}.vdi` + дамп пам'яті `.sav` на 1,21 ГБ).
  - Експортований OVA-файл: `/Users/taras/VMs/oslabs/export/Zahorskyi_Ubuntu.ova` (4,84 ГБ).
  - Спільна тека: `/Users/taras/VMs/oslabs/lab3_shared/` (в Ubuntu монтується у `/home/taras/lab3_shared`).

💻 ПОЯСНЕННЯ КОДУ, ФУНКЦІЙ ТА ПАРАМЕТРІВ

```
cd "/Users/taras/Desktop/unik/Operation Systme" && ./start_defense.sh 3
```

Повна підготовка ЛР3: відкриває PDF, папки ВМ та OVA у Finder, виводить VBoxManage інфо і вміст спільної теки.

```
VBoxManage startvm 'Zahorskyi_Ubuntu' --type gui
```

Запускає віртуальну машину Zahorskyi_Ubuntu у графічному вікні (запускай, якщо попросять показати вживу).

20 Построковий розбір усіх 20 команд звіту (Шпаргалка)

Команди VBoxManage (macOS) та bash-команди Linux Ubuntu зі звіту ЛР 3

ЩО КАЗАТИ ВИКЛАДАЧУ (СЦЕНАРІЙ ДОПОВІДІ)

У лабораторній роботі № 3 я налаштовував віртуальну машину як через GUI, так і через `VBoxManage`, а всі перевірки всередині гостьової Ubuntu 24.04 виконував стандартними утилітами Linux у терміналі bash. Ось построковий розбір усіх 20 команд звіту.

ПОЯСНЕННЯ КОДУ, ФУНКЦІЙ ТА ПАРАМЕТРІВ

```
1. VBoxManage createvm --name Zahorskyi_Ubuntu --ostype Ubuntu24_LTS_arm64 --register --default
```

Функція: створює ВМ з профілем Ubuntu 24.04 ARM64. **Параметри:** `--register` (реєструє в VirtualBox), `--default` (створює базовий набір контролерів за замовчуванням).

```
2. VBoxManage modifyvm Zahorskyi_Ubuntu --memory 4096 --cpus 4 --vram 128 --clipboard-mode bidirectional --drag-and-drop bidirectional
```

Функція: виділяє 4 ГБ ОЗП, 4 ядра CPU, 128 МБ відеопам'яті та вмикає двонаправлений буфер обміну і drag'n'drop.

```
3. VBoxManage modifyvm Zahorskyi_Ubuntu --nic1 nat --usb-xhci on --boot1 disk --boot2 dvd --boot3 none --boot4 none
```

Функція: вмикає мережевий адаптер 1 у режимі `nat`, контролер USB 3.0 (`--usb-xhci on`) та задає чергу завантаження: 1) диск, 2) DVD.

```
4. VBoxManage createmedium disk --filename Zahorskyi_Ubuntu.vdi --size 25600 --format VDI
```

Функція: створює динамічний віртуальний диск формату VDI обсягом 25 600 МБ (25 ГБ).

```
5. VBoxManage storageattach Zahorskyi_Ubuntu --storagectl SATA --port 0 --type hdd --medium Zahorskyi_Ubuntu.vdi
```

Функція: підключає основний VDI-диск до порту 0 контролера SATA (AHCI). Ми спеціально перенесли диск із VirtIO-SCSI на SATA AHCI, щоб уникнути помилки `VERR_SSM_LOAD_CONFIG_MISMATCH` при відновленні збереженого стану.

```
6. VBoxManage storageattach Zahorskyi_Ubuntu --storagectl SATA --port 1 --type dvddrive --medium ubuntu-24.04.5-desktop-arm64.iso
```

Функція: монтує інсталяційний ISO-образ Ubuntu 24.04.5 у віртуальний оптичний привід на порту 1 контролера SATA.

7. `VBoxManage sharedfolder add Zahorskyi_Ubuntu --name lab3_shared --hostpath ~/VMs/oslabs/lab3_shared --automount`

Функція: додає спільну теку хоста macOS з автоматичним монтуванням у гостьовій Ubuntu (`--automount`).

8. `VBoxManage storagectl Zahorskyi_Ubuntu --name USB --add usb && VBoxManage storageattach ... USB_flash.vdi`

Функція: створює контролер носіїв USB і підключає до нього віртуальну флешку `USB_flash.vdi` обсягом 1 ГБ.

9. `VBoxManage unattended install Zahorskyi_Ubuntu --iso=ubuntu-24.04.5-desktop-arm64.iso --user=taras --locale=uk_UA --install-additions --start-vm=gui`

Функція: запускає автоматичне встановлення Ubuntu з українською локаллю, створює користувача `taras` і автоматично компілює та ставить модулі Guest Additions 7.2.22.

10. `[Ubuntu] lsmod | grep vbox && VBoxControl --version`

Функція: перевіряє завантажені модулі ядра Linux (рис. 2). Показує модулі `vboxsf` (спільні теки) та `vboxguest` (базовий драйвер інтеграції) версії `7.2.22r175245`.

11. `[Ubuntu] systemctl status vboxadd-service --no-pager && ps -ef | grep VBoxClient`

Функція: перевіряє активність системного демона `vboxadd-service` (`active (running)`) і процесів сесії користувача `VBoxClient --clipboard`, `--draganddrop`, `--vmsvga-session`.

12. `[Ubuntu] ip -4 addr show && ip route`

Функція: мережева діагностика пакета iproute2 (рис. 5). `ip -4 addr` показує IPv4-адресу `10.0.2.15/24` на інтерфейсі `enp0s8`; `ip route` показує маршрут за замовчуванням `default via 10.0.2.2`.

13. `[Ubuntu] ping -c 3 google.com && curl -sI https://ubuntu.com | head -5`

Функція: перевірка виходу в Інтернет через NAT. Ключ `-c 3` відправляє 3 ICMP-пакети (0% packet loss), а `curl -sI` (`-I` — тільки HTTP-заголовки) повертає `HTTP/2 200`.

14. `[Ubuntu] lsusb && lsblk -o NAME,SIZE,TYPE,TRAN,FSTYPE,LABEL,MOUNTPOINT`

Функція: перевірка USB-флешки (рис. 6). `lsusb` бачить `VirtualBox USB Harddisk` на шині USB 3.0; `lsblk` показує блоковий пристрій `sdb` з транспортом `TRAN=usb`, ФС `vfat`, міткою `USB_FLASH` і точкою монтування `/media/taras/USB_FLASH`.

15. [Ubuntu] echo 'Запис на USB-флешку з Ubuntu (Загорський Т., ІП3-31)' > /media/taras/USB_FLASH/usb_test.txt && cat ...

Функція: перевіряє реальний запис і читання даних на змонтованому FAT32 розділі віртуальної USB-флешки.

16. [Ubuntu] lsblk /dev/sr0 -o NAME,SIZE,TYPE,FSTYPE,LABEL,MOUNTPOINT && ls '/media/taras/Ubuntu 24.04.5 LTS arm64'

Функція: перевірка CD/DVD приводу (рис. 7). Показує пристрій `sr0` типу `rom` з файловою системою `iso9660` обсягом 3,7 ГБ та каталоги інсталятора (`boot` , `casper` , `EFI` , `pool`).

17. [Ubuntu] df -hT ~/lab3_shared && cat ~/lab3_shared/from_host_macOS.txt && echo '...' > ~/lab3_shared/from_guest_ubuntu.txt

Функція: перевірка спільної теки (рис. 8). Ключ `-T` у `df -hT` показує тип файлової системи `vboxsf` , після чого виконується двостороннє читання і запис файлів між macOS та Ubuntu.

18. VBoxManage snapshot Zahorskyi_Ubuntu take 'Після налаштування' && ls -lh '~/VirtualBox VMs/Zahorskyi_Ubuntu/Snapshots'

Функція: створює онлайн-знімок стану працюючої ВМ (рис. 12). У теці `Snapshots` з'являються два диференційні диски `{UUID}.vdi` та файл стану пам'яті `.sav` (1,21 ГБ).

19. VBoxManage controlvm Zahorskyi_Ubuntu savestate && VBoxManage startvm Zahorskyi_Ubuntu

Функція: зберігає поточний стан ВМ на диск (стан «Збережено», створюється другий `.sav` на 1,21 ГБ, рис. 13–14), а `startvm` миттєво будить ВМ з видаленням тимчасового `.sav` .

20. VBoxManage export Zahorskyi_Ubuntu -o Zahorskyi_Ubuntu.ova --manifest && tar -tvf Zahorskyi_Ubuntu.ova

Функція: пакує вимкнену ВМ у переносний архів OVF `Zahorskyi_Ubuntu.ova` (4,84 ГБ) з маніфестом хеш-сум `--manifest` (рис. 15).

📊 ПОЯСНЕННЯ РЕЗУЛЬТАТІВ (ЧОМУ ТАК ВИЙШЛО)

Підсумок по коду: Ці 20 команд повністю покривають адміністрування гіпервізора на хості macOS та перевірку модулів ядра, мережі, блокових пристроїв і файлових систем у Linux Ubuntu.

👉 ІНСТРУКЦІЯ ДЛЯ ТЕБЕ (ЩО НАТИСНУТИ / ПОКАЗАТИ)

Гортай нижче по сторінках звіту під час доповіді викладачу.

1 Титульна, Мета, Середовище та Створення ВМ Zahorskyi_Ubuntu

Сторінки 1–4 звіту · Розділи 1–3 · Рисунок 1

🎙️ ЩО КАЗАТИ ВИКЛАДАЧУ (СЦЕНАРІЙ ДОПОВІДІ)

Добрий день! Представляю лабораторну роботу № 3 «Використання Oracle VM VirtualBox» (варіант 6). Мета роботи — створити віртуальну машину **Zahorskyi_Ubuntu**, встановити Linux Ubuntu, налаштувати гостьові доповнення, спільні ресурси, дослідити знімки, збереження стану та експорт ВМ.

Оскільки мій MacBook Pro працює на процесорі Apple M1 Pro (ARM64), я використав VirtualBox 7.2.22 для Apple Silicon та офіційний образ **Ubuntu 24.04.5 LTS Desktop для arm64**. Машині виділено 4 ГБ ОЗП, 4 ядра процесора, 128 МБ відеопам'яті та динамічний диск VDI на 25 ГБ (оскільки рекомендованих у методичці 12 ГБ для сучасної Ubuntu 24.04 Desktop недостатньо). Усі параметри показано у головному вікні менеджера на рисунку 1.

📊 ПОЯСНЕННЯ РЕЗУЛЬТАТІВ (ЧОМУ ТАК ВИЙШЛО)

Пояснення результатів (Рис. 1): У вікні VirtualBox Manager видно створену машину `Zahorskyi_Ubuntu` з увімкненим EFI, порядком завантаження «Жорсткий диск → Оптичний диск», контролером SATA AHCI, мережею NAT, контролером USB xHCI та спільною текою `lab3_shared`.

👉 ІНСТРУКЦІЯ ДЛЯ ТЕБЕ (ЩО НАТИСНУТИ / ПОКАЗАТИ)

Що показати: На сторінці 3 — блок команд `VBoxManage modifyvm`, на сторінці 4 — Рисунок 1.

5 Гостьові доповнення, Буфер обміну, Drag'n'Drop та Мережа NAT

Сторінки 5–7 звіту · Розділи 3–5 · Рисунки 2–5

🎙️ ЩО КАЗАТИ ВИКЛАДАЧУ (СЦЕНАРІЙ ДОПОВІДІ)

На рисунку 2 у терміналі Ubuntu перевірено роботу **Guest Additions 7.2.22**: команда `lsmod | grep vbox` показує завантажені в ядро Linux модулі `vboxguest` і `vboxsf`, служба `vboxadd-service` активна, а процеси `VBoxClient` обслуговують буфер обміну, drag'n'drop та автозміну роздільної здатності екрана.

На рисунках 3 і 4 протестовано двонаправлений буфер обміну: український текст із macOS вставився у текстовий редактор Ubuntu зі збереженням кирилиці, а скопійований в Ubuntu рядок одразу з'явився у вікні «Буфер обміну» Finder на Mac. Як і в ЛР2, **drag'n'drop** увімкнений і служба `VBoxClient --draganddrop` запущена, але сам хост macOS у VirtualBox 7.2 не підтримує перетягування файлів з Finder.

На рисунку 5 перевірено мережу **NAT**: на інтерфейсі `enp0s8` отримано адресу `10.0.2.15/24`, шлюз `10.0.2.2`, `ping -c 3 google.com` пройшов з 0% втрат, а `curl -sI https://ubuntu.com` повернув статус `HTTP/2 200`.

📊 ПОЯСНЕННЯ РЕЗУЛЬТАТІВ (ЧОМУ ТАК ВИЙШЛО)

Пояснення результатів (Рис. 2–5): На відміну від Windows ARM64 у ЛР2, ядро Linux 7.0 в Ubuntu ARM64 має вбудований драйвер мережевої карти Intel PRO/1000 MT Desktop (`82540EM`, драйвер `e1000`), тому мережа NAT в Ubuntu запрацювала одразу «з коробки» без додаткових драйверів.

👉 ІНСТРУКЦІЯ ДЛЯ ТЕБЕ (ЩО НАТИСНУТИ / ПОКАЗАТИ)

Що показати: На рис. 2 — вивід `lsmod` та `VBoxClient`; на рис. 3–4 — буфер обміну; на рис. 5 — `10.0.2.15/24` і `HTTP/2 200`.

8 USB-накопичувач, CD/DVD, Спільна тека та Завантаження з DVD

Сторінки 8–10 звіту · Розділи 6–9 · Рисунки 6–10

🎙️ ЩО КАЗАТИ ВИКЛАДАЧУ (СЦЕНАРІЙ ДОПОВІДІ)

На рисунку 6 перевірено роботу USB: до USB-контролера xHCI підключено віртуальний накопичувач `USB_flash.vdi` (1 ГБ). В Ubuntu утиліта `lsusb` бачить його як `VirtualBox USB Harddisk` на шині USB 3.0, а `lsblk` показує пристрій `sdb` з типом транспорту `TRAN=usb`, змонтований у `/media/taras/USB_FLASH`. Я успішно записав і прочитав на ньому тестовий файл.

На рисунку 7 показано оптичний привід `/dev/sr0` з файловою системою `iso9660`, куди змонтований інсталяційний DVD Ubuntu. На рисунку 8 перевірено спільну теку `~/lab3_shared` (тип файлової системи `vboxsf`): файли `from_host_macOS.txt` і `from_guest_ubuntu.txt` вільно читаються і створюються в обидва боки.

На рисунках 9 і 10 перевірено можливість завантаження з CD/DVD: під час старту ВМ клавішею **Esc** я викликав меню **UEFI Boot Manager**, обрав `UEFI VBOX CD-ROM` і завантажився у меню GRUB інсталяційного диска «Try or Install Ubuntu».

📊 ПОЯСНЕННЯ РЕЗУЛЬТАТІВ (ЧОМУ ТАК ВИЙШЛО)

Пояснення результатів (Рис. 6–10): Файлова система `vboxsf` (VirtualBox Shared Folders) працює як віртуальна мережева файлова система поверх гіпервізорного виклику HGCM (Host-Guest Communication Manager), дозволяючи монтувати директорії APFS хоста macOS прямо у дерево каталогів Linux.

👉 ІНСТРУКЦІЯ ДЛЯ ТЕБЕ (ЩО НАТИСНУТИ / ПОКАЗАТИ)

Що показати: На рис. 6 стовпець `TRAN = usb` у виводі `lsblk`; на рис. 8 — тип `vboxsf`; на рис. 9–10 — меню UEFI та GRUB.

11 Таблиця 1, Файли ВМ, Знімок, Збереження стану та Експорт OVA

Сторінки 11–15 звіту · Розділи 10–13 · Рисунки 11–15

🎙️ ЩО КАЗАТИ ВИКЛАДАЧУ (СЦЕНАРІЙ ДОПОВІДІ)

У таблиці 1 на сторінках 11–12 підсумовано всі параметри основної ОС (macOS 27.2, 16 ГБ ОЗП, APFS) та гостьової ОС (Ubuntu 24.04.5, 4 ГБ ОЗП, VDI 25 ГБ ext4).

У розділах 11–13 (рисунки 11–15) досліджено супровідні файли у папці `~/VirtualBox VMs/Zahorskyi_Ubuntu`: конфігурацію `.vbox`, диск `Zahorskyi_Ubuntu.vdi` (займає лише 9,3 ГБ з 25 ГБ), `.nvram`, папки `Logs` та `Snapshots`.

Після створення онлайн-знімка «Після налаштування» у `Snapshots` з'явилися два диференційні диски `{UUID}.vdi` та дамп пам'яті `.sav` на 1,21 ГБ (рисунок 12). Під час збереження стану (`savestate`) машина перейшла у статус «Збережено» і створився другий файл `.sav` на 1,21 ГБ (рисунки 13–14), який автоматично видалився після відновлення роботи. Тут я також вирішив технічну проблему VirtualBox 7.2 на ARM: зі стандартним контролером VirtIO-SCSI збережений стан видавав помилку `VERR_SSM_LOAD_CONFIG_MISMATCH`, тому я переніс диск на контролер **SATA AHCI**, де `savestate` працює бездоганно.

На рисунку 15 показано готовий файл експорту `Zahorskyi_Ubuntu.ova` (4,84 ГБ).

📊 ПОЯСНЕННЯ РЕЗУЛЬТАТІВ (ЧОМУ ТАК ВИЙШЛО)

Пояснення результатів (Рис. 11–15): Чому файл `.sav` в Ubuntu важить 1,21 ГБ, а у Windows 2,1 ГБ при однакових 4 ГБ ОЗП? Тому що VirtualBox при збереженні стану записує у `.sav` лише реально зайняті (ненульові) сторінки фізичної пам'яті гостя зі стисненням LZ4, а Linux споживає вдвічі менше ОЗП, ніж Windows 11.

👉 ІНСТРУКЦІЯ ДЛЯ ТЕБЕ (ЩО НАТИСНУТИ / ПОКАЗАТИ)

Що показати: Таблицю 1 на стор. 11, теку `Snapshots` на рис. 12–14 та файл `Zahorskyi_Ubuntu.ova` на рис. 15.

Висновки та Контрольні питання (ЛР 3)

Сторінки 15–17 звіту · 8 питань самоперевірки

ЩО КАЗАТИ ВИКЛАДАЧУ (СЦЕНАРІЙ ДОПОВІДІ)

Висновки: Створено і налаштовано віртуальну машину `Zahorskyi_Ubuntu` з Ubuntu 24.04.5 LTS ARM64. Завдяки модулям ядра Guest Additions (`vboxguest`, `vboxsf`) працюють двонаправлений буфер обміну, спільна тека та авторозмір екрана, а також перевірені мережа NAT, віртуальна USB-флешка та завантаження з DVD через UEFI Boot Manager. Досліджено механізм диференційних дисків знімків, збереження стану пам'яті `.sav` та експорт у формат OVF/OVA. Дякую за увагу!

1. Які переваги та недоліки використання ВМ? +

2. Які можливості надають гостьові доповнення ВМ в Linux? +

3. На що впливає об’єм оперативної пам’яті, виділеної для ВМ? +

4. В чому полягає різниця між динамічним і фіксованим ВЖД? +

5. Що таке знімки ВМ і де вони знаходяться? +

6. Яким чином і для чого здійснюється експорт та імпорт ВМ? +

7. Які типи мережевих підключень дозволяє налаштовувати ВМ? +

8. Які папки та файли створюються для супроводу ВМ? +

Підготовка на Mac та Контекст роботи (ЛР 4)

Варіант 6 (Paint) — де лежать файли .blg/.gif/.csv і як показати наживо

🚀 ШПАРГАЛКА ПІДГОТОВКИ ТА КОНТЕКСТ (ДЛЯ ТЕБЕ НА MAC)

Контекст того, як зроблена робота: Твій варіант — **№ 6**. За таблицею варіантів методички досліджуваний процес для варіанта 6 — **графічний редактор (Paint / `mspaint.exe`)**. Робота виконана у віртуальній машині з Windows 11 Pro ARM64 (4 ядра CPU, 4 ГБ ОЗП). У консолі **Performance Monitor (`perfmon.exe`)** була створена група збирачів даних **Lab** (шаблон лежить у `~/VMs/oslabs/win/lab4/Lab.xml`, а скрипт збору — у `collect.ps1`).

- **1. Автоматичний запуск всього одною командою в Терміналі (macOS):** `cd "/Users/taras/Desktop/unik/Operation Systme" && ./start_defense.sh 4` *Цей скрипт сам відкриє PDF-звіт ЛР4, відкриє у Finder папку `lab4/результати_PerfMon` з усіма 12 файлами (4 двійкові логи `.blg`, 4 експортовані таблиці `.csv` і 4 графіки `.gif`), а прямо в терміналі Mac порахує точні середні та максимальні значення з CSV-файлів!*
- **2. Що лежить у папці `lab4/результати_PerfMon/` (обов'язкова вимога методички):**
  - Без взаємодії (1 хв простою): `idle_modes.blg`, `idle_modes.gif`, `idle_interrupts.blg`, `idle_interrupts.gif` (+ `.csv`).
  - З навантаженням у Paint (1 хв малювання): `paint_modes.blg`, `paint_modes.gif`, `paint_interrupts.blg`, `paint_interrupts.gif` (+ `.csv`).
- **3. Як показати наживо у Windows 11 (якщо викладач попросить відкрити PerfMon або Task Manager):**
  - У запущеній ВМ `Zahorskyi_Win` натисни **Win + R** → введи `perfmon` → Enter (відкриється Монітор продуктивності).
  - Щоб показати «Час ядра» у Диспетчері завдань: натисни **Ctrl + Shift + Esc** (або Win+R → `taskmgr`) → вкладка **Продуктивність (Performance)** → **ЦП (CPU)** → клікни **правою кнопкою миші** по графіку → увімкни галочку **«Показати час ядра» (Show kernel times)**.

💻 ПОЯСНЕННЯ КОДУ, ФУНКЦІЙ ТА ПАРАМЕТРІВ

```
cd "/Users/taras/Desktop/unik/Operation Systme" && ./start_defense.sh 4
```

Повна підготовка ЛР4: відкриває PDF, папку результати_PerfMon (.blg, .gif, .csv) та розраховує таблиці 3 і 4 у терміналі.

20 Построковий розбір усіх 20 лічильників і команд звіту (Шпаргалка)

Усі 10 лічильників PerfMon та 10 команд керування збором даних (logman, relog, Lab.xml)

ЩО КАЗАТИ ВИКЛАДАЧУ (СЦЕНАРІЙ ДОПОВІДІ)

У лабораторній роботі № 4 використовувалися 10 системних лічильників продуктивності Windows (6 у збирачі «Режими процесора» та 4 у збирачі «Переривання»), а також утиліти командного рядка `perfmon`, `logman` та `relog` для автоматизації збору та розрахунку статистики. Ось построковий розбір усіх 20 елементів.

ПОЯСНЕННЯ КОДУ, ФУНКЦІЙ ТА ПАРАМЕТРІВ

1. `\Processor(_Total)\% Processor Time`

Лічильник Таблиці 1: відсоток часу, протягом якого всі ядра процесора (`_Total`) виконували не-холості потоки (сума `% User Time + % Privileged Time`). Нормальний поріг — до **85%**.

2. `\Processor(_Total)\% Idle Time`

Лічильник Таблиці 1: відсоток часу, коли процесор простоював у стані очікування (без взаємодії у нас = **95,031%**, під час малювання в Paint впав до **81,835%**).

3. `\Processor(_Total)\% Privileged Time`

Лічильник Таблиці 1: частка часу роботи всіх ядер ЦП у привілейованому **режимі ядра (Kernel Mode, Ring 0 / EL1)** — системні виклики, драйвери, графіка GDI/DirectX, обробка переривань.

4. `\Processor(_Total)\% User Time`

Лічильник Таблиці 1: частка часу роботи всіх ядер ЦП у **режимі користувача (User Mode, Ring 3 / EL0)** — виконання коду самої програми `mspaint.exe` (у Paint зросла з 2,263% до **10,884%**).

5. `\Process(Idle)\% Privileged Time (Масштаб 0.1)`

Лічильник Таблиці 1: час системного псевдопроцесу «Бездіяльність системи» (`Idle`) у режимі ядра. Оскільки у ВМ 4 ядра (максимум 400%), встановлено масштаб `0.1` (без взаємодії = **381,551%**, з Paint = **331,141%**).

6. `\Process(Idle)\% User Time`

Лічильник Таблиці 1: час процесу `Idle` у режимі користувача. Завжди дорівнює **0,000%**, бо потоки Idle існують виключно всередині ядра ОС (виконують інструкцію зупинки ядра HLT/WFI).

`7. \Processor(_Total)\Interrupts/sec (Масштаб 0.01)`

Лічильник Таблиці 2: середня кількість апаратних переривань (IRQ), які процесор обробив за секунду. При малюванні мишею в Paint зросла у 19 разів — зі **179,78** до **3 478,23 за секунду**!

`8. \Processor(_Total)\% Interrupt Time`

Лічильник Таблиці 2: відсоток часу процесора, витрачений на виконання процедур обробки переривань (ISR) у режимі ядра. У нас максимум **0,157%** і **1,090%** (критичний поріг — понад **20–30%**).

`9. \System\System Calls/sec (Масштаб 0.001)`

Лічильник Таблиці 2: частота звернень програм до системних викликів ядра ОС за секунду. У Paint зросла в 11,7 раза — з **2 616,55** до **30 745,73 викликів/с** (масштаб `0.001` для відображення на шкалі 0–100).

`10. \System\Exception Dispatches/sec`

Лічильник Таблиці 2: кількість оброблених ядром виключних ситуацій процесора (exceptions, page faults) за секунду (у нас **1,183** в простої та **4,424** у Paint).

`11. perfmon.exe /sys`

Команда Windows: запускає консоль «Монітор продуктивності» (Performance Monitor). Ключ `/sys` відкриває одразу окремий монітор у реальному часі.

`12. <DataCollectorSet><Duration>60</Duration><RootPath>C:\PerfLogs\Lab</RootPath>`

Параметри XML-шаблону `Lab.xml`**:** задає автоматичну умову зупинки групи збирачів рівно через **60 секунд (1 хвилину)** і кореневу папку збереження логів.

`13. <SampleInterval>5</SampleInterval><LogFileFormat>3</LogFileFormat><LogOverwrite>—1</LogOverwrite>`

Параметри збирача: інтервал вибірки `5` секунд (дає 12 точок за 1 хвилину), формат файлу `3` (двійковий журнал `.blg` — Binary Log), режим `LogOverwrite=-1` (перезаписувати старий файл).

`14. logman import -n Lab -xml C:\lab\Lab.xml`

Команда адміністрування: імпортує групу збирачів даних з іменем `-n Lab` із конфігураційного XML-файлу `-xml` у розділ User Defined.

`15. logman start Lab && logman query Lab`

Команда: `logman start Lab` запускає фоновий запис обох збирачів («Режими процесора» і «Переривання»); `logman query Lab` перевіряє статус (`Running` або `Stopped`).

```
16. mspaint.exe
```

Досліджуваний процес (Варіант 6): стандартний растровий графічний редактор Windows Paint, у якому ми активно малювали пензлем, фігурами і заливкою протягом 60 секунд.

```
17. Copy-Item C:\PerfLogs\Lab\modes.blg C:\PerfLogs\Lab\paint_modes.blg -Force
```

Команда PowerShell (`collect.ps1`): після завершення 1-хвилинного сеансу копіює бінарні логи під осмисленими іменами `idle_modes.blg` , `idle_interrupts.blg` , `paint_modes.blg` , `paint_interrupts.blg` .

```
18. relog C:\PerfLogs\Lab\paint_modes.blg -f csv -o C:\PerfLogs\Lab\paint_modes.csv -y
```

Утиліта конвертації журналів `relog` : перетворює двійковий журнал `.blg` у текстову таблицю CSV (`-f csv`) у вихідний файл `-o` без запиту підтвердження (`-y`) для точного обчислення середніх та максимумів.

```
19. PerfMon GUI → Ctrl+Q (Properties → Data → Scale / Width) → Save Image As (.gif)
```

Форматування графіка в PerfMon: виклик властивостей графіка, вибір максимальної товщини ліній, налаштування масштабів (0.1, 0.01, 0.001) і збереження растрового знімка у формат `.gif` .

```
20. taskmgr.exe → Вкладка «Продуктивність» → ЦП → ПКМ → «Відображати час ядра»
```

Команда Диспетчера завдань: вмикає на графіку завантаження ЦП окреме затінення для часу роботи в привілейованому режимі ядра (Kernel Time, рис. 8).

📊 ПОЯСНЕННЯ РЕЗУЛЬТАТІВ (ЧОМУ ТАК ВИЙШЛО)

Підсумок по коду і лічильниках: Інтервал вибірки 5 секунд за 60 секунд спостереження дає рівно **12 відліків** у кожному файлі `.blg` / `.csv` , за якими обчислені всі цифри в Таблицях 3 і 4.

👉 ІНСТРУКЦІЯ ДЛЯ ТЕБЕ (ЩО НАТИСНУТИ / ПОКАЗАТИ)

Якщо викладач спитає, звідки взялися точні цифри до тисячних у Таблиці 3 — скажи: «Я експортував двійкові файли `.blg` у формат CSV штатною утилітою Windows `relog` , де записані всі 12 вибірок з високою точністю, і вони повністю збігаються з полем Average під графіком PerfMon».

1 Мета, Група збирачів даних Lab та Процес збору (Варіант 6 — Paint)

Сторінки 1–5 звіту · Розділи 1–3 · Рисунки 1–3

🎙️ ЩО КАЗАТИ ВИКЛАДАЧУ (СЦЕНАРІЙ ДОПОВІДІ)

Добрий день! Представляю лабораторну роботу № 4 «Дослідження режимів роботи ОС та обробки переривань» (мій варіант — **№ 6**, графічний редактор **Paint**).

Мета роботи — за допомогою системної консолі **Performance Monitor** (`perfmon.exe`) дослідити розподіл часу процесора між режимом користувача (User Mode) і режимом ядра (Kernel / Privileged Mode), а також інтенсивність апаратних переривань і системних викликів.

У розділі «Групи збирачів даних → Особисті (User Defined)» я створив групу **Lab** із двома збирачами (рисунок 1): «Режими процесора» (6 лічильників Таблиці 1) та «Переривання» (4 лічильники Таблиці 2). Для обох задано інтервал вибірки **5 секунд**, двійковий формат журналу **Binary (.blg)**, режим запису **Overwrite (із заміною)** та автоматичну зупинку через **1 хвилину** (рисунок 2).

Збір проводився двічі по 1 хвилині: перший раз у повному спокої системи без взаємодії, другий раз — під час інтенсивного малювання пензлем, фігурами і заливкою у графічному редакторі **Paint** (рисунок 3).

📊 ПОЯСНЕННЯ РЕЗУЛЬТАТІВ (ЧОМУ ТАК ВИЙШЛО)

Пояснення масштабування графіків (стор. 4): Оскільки шкала графіка PerfMon становить від 0 до 100, деякі лічильники довелося масштабувати у вкладці Data: для `Process(Idle): % Privileged Time` встановлено масштаб **0,1** (бо на 4-ядерному ЦП сумарний час простою всіх ядер сягає 400%), для `Interrupts/sec` — масштаб **0,01** (значення ~3500/с на графіку виглядає як 35), а для `System Calls/sec` — **0,001** (значення ~30 700/с на графіку виглядає як 30,7).

👉 ІНСТРУКЦІЯ ДЛЯ ТЕБЕ (ЩО НАТИСНУТИ / ПОКАЗАТИ)

Що показати: На рис. 1 — два збирачі у групі Lab; на рис. 2 — вікно властивостей (Binary, Sample interval: 5 seconds); на рис. 3 — малювання у Paint.

6 Аналіз графіків: Режими процесора та Обробка переривань

Сторінки 6–7 звіту · Розділи 4–5 · Рисунки 4–7

ЩО КАЗАТИ ВИКЛАДАЧУ (СЦЕНАРІЙ ДОПОВІДІ)

На сторінці 6 (рисунки 4 і 5) порівнюються режими роботи процесора:

• **Без взаємодії (рис. 4):** синя лінія `% Idle Time` тримається біля **95%**, а червона лінія `Process(Idle): % Privileged Time` (у масштабі 0,1) — на рівні **38,1** (тобто 381,5% із 400% на 4 ядрах). Корисне завантаження ЦП становить лише 4,6%.

• **Під час малювання в Paint (рис. 5):** загальне завантаження ЦП (зелена лінія `% Processor Time`) зросло до **17,2%** (у піках до 28,5%). Найголовніше: блакитна лінія `% User Time` (**10,88%**) піднялася значно вище за жовту лінію `% Privileged Time` (**6,32%**).

На сторінці 7 (рисунки 6 і 7) показано обробку переривань:

• **Без взаємодії (рис. 6):** усі лінії лежать майже на нулі (180 переривань/с та 2616 системних викликів/с).

• **У Paint (рис. 7):** зелена лінія `Interrupts/sec` підскочила у **19 разів** (до **3 478 переривань/с**), а жовта лінія `System Calls/sec` — майже у **12 разів** (до **30 745 викликів/с**). При цьому червона лінія `% Interrupt Time` залишилася внизу (лише 0,61%).

ПОЯСНЕННЯ РЕЗУЛЬТАТІВ (ЧОМУ ТАК ВИЙШЛО)

Глибоке пояснення результатів для викладача:
1) Чому в простої `Process(Idle)` має 381% саме у **Privileged Time** і 0% у User Time? Тому що `System Idle Process` (PID 0) — це системні потоки ядра (по одному на кожне з 4 ядер), які виконують привілейовану інструкцію енергозбереження процесора (`WFI` на ARM64 / `HLT` на x86), що дозволена тільки в режимі ядра!
2) Чому в Paint так сильно виросли і User Time, і Privileged Time, і Interrupts/sec? Кожен мікрорух миші генерує апаратне переривання від USB-контролера xHCI (звідси 3478 переривань/с). У **User Mode** Paint рахує геометрію кривої пензля, алгоритм заливки та змінює пікселі бітмапу в пам'яті (10,88%). А щоб відобразити кожен мазок на екрані, Paint робить тисячі системних викликів до графічної підсистеми ядра `win32kfull.sys` / GDI та

драйвера відеокарти (звідси 30 745 System Calls/sec і ріст Privileged Time до 6,32%).

👉 **ІНСТРУКЦІЯ ДЛЯ ТЕБЕ (ЩО НАТИСНУТИ / ПОКАЗАТИ)**

Що показати: На рис. 5 вкажи на блакитну лінію (User Time) над жовтою (Privileged Time); на рис. 7 — на високі зелену та жовту криві під час малювання.

8 Таблиці 3 і 4 (Чисельні результати) та Диспетчер завдань

Сторінки 8–9 звіту · Розділи 6–7 · Таблиці 3–4 · Рисунок 8

🎙️ ЩО КАЗАТИ ВИКЛАДАЧУ (СЦЕНАРІЙ ДОПОВІДІ)

У Таблиці 3 на сторінці 8 зведені точні середні значення всіх 10 лічильників:

- `% Processor Time`: зріс із **4,612%** до **17,215%**;
- `% Idle Time`: знизився з **95,031%** до **81,835%**;
- `% Privileged Time`: зріс із **2,348%** до **6,319%**;
- `% User Time`: зріс у 4,8 раза — з **2,263%** до **10,884%**;
- `Interrupts/sec`: зросли зі **179,784** до **3 478,229**;
- `System Calls/sec`: зросли з **2 616,546** до **30 745,728**.

У Таблиці 4 перевірено граничні пороги перевантаження системи: максимальне значення `% Processor Time` склало **17,332%** у простої та **28,540%** у Paint (при допустимому порозі **85%**), а `% Interrupt Time` — максимум **0,157%** та **1,090%** (при порозі **20%**). Тривалість перевищення порогів — **0 секунд**.

На рисунку 8 додатково показано вкладку «Продуктивність → ЦП» у Диспетчері завдань із увімкненою опцією «Показати час ядра», де темно-синя область наочно показує частку Kernel Mode під час роботи Paint.

📊 ПОЯСНЕННЯ РЕЗУЛЬТАТІВ (ЧОМУ ТАК ВИЙШЛО)

Пояснення результатів Таблиці 4: Оскільки `% Processor Time < 85%` (28,54%) і `% Interrupt Time < 20%` (1,09%), вузьких місць (bottlenecks) процесора немає, а драйвери пристроїв вводу-виводу працюють абсолютно справно без «шторму переривань».

👉 ІНСТРУКЦІЯ ДЛЯ ТЕБЕ (ЩО НАТИСНУТИ / ПОКАЗАТИ)

Що показати: Таблиці 3 і 4 на сторінці 8 та Рисунок 8 на сторінці 9.

Висновки та Контрольні питання (ЛР 4)

Сторінки 9–11 звіту · 6 питань самоперевірки

ЩО КАЗАТИ ВИКЛАДАЧУ (СЦЕНАРІЙ ДОПОВІДІ)

Висновки: За допомогою Performance Monitor ми експериментально підтвердили поділ роботи ОС на два режими. Без взаємодії 95% часу процесор проводить у стані простою, який обліковується ядерним процесом `Idle` у режимі ядра. При запуску прикладної програми Paint основні обчислення (малювання фігур і растру) виконуються в непривілейованому **режимі користувача** (10,88%), а ввід координат миші та вивід графіки на екран передаються через системні виклики (~30,7 тис./с) та апаратні переривання (~3,5 тис./с) у **режим ядра** (6,32%). Граничні пороги (85% ЦП та 20% переривань) ніде не перевищені. Дякую, готовий відповідати на питання!

1. Характеристики режиму ядра (Kernel / Privileged Mode).

2. Для чого використовується процес Idle в ОС Windows?

3. Як можна прослідкувати за часом роботи у режимі користувача та ядра за допомогою Task Manager?

4. Які можливості утиліти Performance Monitor з оперативного спостереження за станом ОС?

5. Які можливості утиліти Performance Monitor зі створення звітів?

6. Які граничні значення лічильників вказують на можливі негаразди в роботі системи?

Підготовка на Mac та Контекст роботи (ЛР 5)

Варіант 6 (N=6, N+1=7, N-1=5) — автоскрипт для Windows cmd.exe

🚀 ШПАРГАЛКА ПІДГОТОВКИ ТА КОНТЕКСТ (ДЛЯ ТЕБЕ НА MAC)

Контекст того, як зроблена робота: Твій варіант — **N = 6** (тому файли називаються `lab6.txt` і `lab7.txt`, порожні файли — `lect6.txt` і `lect5.txt`, у прихованому каталозі `.hidden` — `user6.txt` і `tree6.txt`, у каталозі `OS` — `result6.txt` та жорсткий зв'язок `links\tree6_h.txt`, а точки монтування — `C:\mount\NTFS_6` і `C:\mount\FAT_6`).

- **1. Автоматичний запуск на Mac одною командою в Терміналі:**
  `cd "/Users/taras/Desktop/unik/Operation Systme" && ./start_defense.sh 5`
  Скрипт відкриє PDF-звіт ЛР5, відкриє папку зі скріншотами, виведе всі команди `lab5_steps.txt` *у термінал і покладе готовий автоскрипт* `run_lab5_auto.bat` *прямо у спільну теку Windows!*
- **2. Як за 2 секунди виконати всю ЛР 5 наживо у Windows 11 (якщо попросить викладач):**
  - У запущеній ВМ `Zahorskyi_Win` натисни **Пуск** → введи `cmd` → натисни **«Запустити від імені адміністратора» (Run as administrator)**. *(Права адміністратора обов'язкові для* `mklink /d`, `diskpart` *і* `mountvol`*!)*
  - Щоб не вводити всі 30 команд руками, просто введи в консоль Адміністратора одну команду:
    `Z:\run_lab5_auto.bat`
    Він сам миттєво створить `C:\source_OS`, `.hidden`, *усі файли, посилання* `SYMLINKD`, `JUNCTION`, *жорсткий зв'язок* `HARDLINK`, *сформує* `result6.txt` *і виведе все на екран!*

💻 ПОЯСНЕННЯ КОДУ, ФУНКЦІЙ ТА ПАРАМЕТРІВ

```
cd '/Users/taras/Desktop/unik/Operation Systme' && ./start_defense.sh 5
```

На Mac: відкриває PDF ЛР5, скріншоти і генерує Z:\run_lab5_auto.bat для гостьової Windows.

```
Z:\run_lab5_auto.bat
```

У Windows (cmd від Адміністратора): автоматично виконує всі кроки ЛР5 для варіанта 6 і виводить результат.

20 Построковий розбір усіх 20 команд cmd.exe зі звіту (Шпаргалка)

Кожна команда звіту ЛР 5, її синтаксис, ключі та призначення

🎙️ ЩО КАЗАТИ ВИКЛАДАЧУ (СЦЕНАРІЙ ДОПОВІДІ)

У лабораторній роботі № 5 для створення і дослідження об'єктів файлових систем NTFS та FAT32 у командному інтерпретаторі `cmd.exe` (від імені адміністратора) використано рівно 20 ключових команд. Ось їх построковий розбір із параметрами.

💻 ПОЯСНЕННЯ КОДУ, ФУНКЦІЙ ТА ПАРАМЕТРІВ

1. `cd C:\ && mkdir source_OS && cd source_OS && mkdir labs lects .hidden`

Функція: переходить у корінь диска `C:\`, створює базовий каталог `source_OS` і три підкаталоги однойменною командою `mkdir` (аналог `md`).

2. `attrib +H .hidden`

Функція: змінює атрибути файлу чи каталогу. **Параметр:** `+H` (встановлює атрибут **Hidden** — прихований; для зняття використовується `-H`).

3. `dir && dir /AH`

Функція: виводить вміст каталогу. Звичайний `dir` не показує приховані об'єкти, а ключ `/AH` (`/A` — фільтр за атрибутом, `H` — Hidden) виводить тільки прихований каталог `.hidden` (рис. 1).

4. `echo lab6 > labs\lab6.txt && echo lab7 > labs\lab7.txt`

Функція: створює текстові файли за допомогою оператора перенаправлення стандартного потоку виводу `>` (перезаписує або створює новий файл і записує туди рядок).

5. `type labs\lab6.txt && type labs\lab7.txt`

Функція: команда `type` виводить текстовий вміст файлу в консоль (рис. 2). Якщо викликати `type lab6.txt` з `C:\source_OS`, система видасть помилку, бо файл лежить у підкаталозі `labs`.

6. `copy con lects\lect6.txt [Ctrl+Z → Enter] && copy con lects\lect5.txt [Ctrl+Z]`

Функція: класичний спосіб створення файлу з потоку консолі (`con` — пристрій клавіатури). Оскільки після вводу команди ми одразу натискаємо комбінацію кінця файлу **Ctrl+Z** (символ `^Z` / EOF), створюються порожні файли довжиною 0 байт (рис. 3).

7. `echo %USERNAME% > .hidden\user6.txt`

Функція: бере ім'я поточного користувача з системної змінної середовища `%USERNAME%` (`Taras`) і записує його у файл `user6.txt` всередині прихованого каталогу.

8. `tree /f /a . > .hidden\tree6.txt`

Функція: будує графічне дерево поточного каталогу (`.`) і зберігає у файл. **Параметри:** `/f` (виводити не лише папки, а й імена файлів у них), `/a` (використовувати ASCII-символи псевдографіки замість розширених символів, щоб у файлі не було кракозябр, рис. 4).

9. `cd %USERPROFILE% && mkdir OS OS\links && cd OS`

Функція: переходить у домашній каталог користувача (`%USERPROFILE%` = `C:\Users\Taras`) і створює каталог `OS` разом із вкладеним каталогом `OS\links` .

10. `mklink /d lects C:\source_OS\lects`

Функція: створює **символічний зв'язок на каталог** (Directory Symbolic Link, тип у `dir` — `<SYMLINKD>`). **Параметр:** `/d` (directory). Може вказувати на відносні шляхи, інші томи та мережеві папки UNC; вимагає прав адміністратора.

11. `mklink /j labs C:\source_OS\labs`

Функція: створює **точку з'єднання каталогів** (Directory Junction, тип у `dir` — `<JUNCTION>`). **Параметр:** `/j` (junction). Працює тільки для локальних каталогів і тільки з абсолютними шляхами (рис. 5–6).

12. `type C:\source_OS\.hidden\user6.txt > result6.txt && echo 6 >> result6.txt`

Функція: формує початок файлу `result6.txt` . Оператор з однією стрілкою `>` створює файл з іменем користувача (`Taras`), а подвійна стрілка `>>` **дописує в кінець** файлу номер варіанта `6` без затирання попередніх рядків.

13. `echo. >> result6.txt && echo. >> result6.txt`

Функція: команда `echo.` (із крапкою впритул до слова echo без пробілу!) виводить порожній рядок (переведення каретки CRLF). Дві команди поспіль додають 2 порожні рядки у `result6.txt` .

14. `type labs\lab6.txt >> result6.txt && type labs\lab7.txt >> result6.txt`

Функція: читає вміст файлів `lab6.txt` та `lab7.txt` **через створений зв'язок-з'єднання** `labs` і дописує їх у кінець `result6.txt` (рис. 7).

15. `mklink /h links\tree6_h.txt C:\source_OS\.hidden\tree6.txt`

Функція: створює **жорсткий зв'язок (Hard Link)** на файл. **Параметр:** `/h` (hardlink). Це не ярлик, а другий рівноправний запис у таблиці MFT (Master File Table) NTFS, що

вказує на ті самі фізичні кластери даних (рис. 8).

16. `fsutil hardlink list links\tree6_h.txt >> result6.txt`

Функція: системна утиліта керування файловою системою `fsutil` з підкомандою `hardlink list` знаходить і виводить усі імена (жорсткі посилання) цього файлу на томі NTFS (`\source_OS\.hidden\tree6.txt` та `\Users\Taras\OS\links\tree6_h.txt`) і дописує їх у `result6.txt` (рис. 9).

17. `cd C:\ && mkdir mount mount\NTFS_6 mount\FAT_6`

Функція: створює порожні каталоги на томі NTFS, які слугуватимуть точками монтування для нових дискових томів (рис. 11).

18. `diskpart /s C:\lab\dp6.txt`

Функція: утиліта керування дисками і розділами. Ключ `/s` виконує пакетний сценарій (`select disk 1` , `clean` , `convert mbr` , `create partition primary size=1000` , `format fs=ntfs label=NTFS_6 quick` , `create partition primary` , `format fs=fat32 label=FAT_6 quick` , рис. 12).

19. `mountvol C:\mount\NTFS_6 \\?\Volume{f8a0692a-0000-0000-0000-010000000000}\`

Функція: монтує відформатований том без літери диска у порожню папку NTFS за його незмінним унікальним ідентифікатором `\\?\Volume{GUID}\` . Ключ `/L` виводить GUID змонтованого тому (рис. 13).

20. `mklink /h C:\mount\FAT_6\... (Перевірка обмежень FAT32 і міжтомних зв'язків)`

Функція: експериментальна перевірка (рис. 15): 1) на FAT32 не працюють ні `mklink /h` (*Incorrect function*), ні `mklink` (*The device does not support symbolic links*); 2) `mklink /h` між різними томами NTFS (`C:` і `NTFS_6`) неможливий (*The system cannot move the file to a different disk drive*); 3) зате символічний зв'язок `mklink` між різними томами працює!

📊 ПОЯСНЕННЯ РЕЗУЛЬТАТІВ (ЧОМУ ТАК ВИЙШЛО)

Підсумок по коду: Ці 20 команд демонструють фундаментальну різницю між таблицями MFT у NTFS (де існують Reparse Points для `SYMLINKD` , `JUNCTION` , `mountvol` та багатоіменні записи файлів для `Hard Link`) і простою таблицею FAT32, де жоден із цих механізмів не підтримується.

👉 ІНСТРУКЦІЯ ДЛЯ ТЕБЕ (ЩО НАТИСНУТИ / ПОКАЗАТИ)

Гортай нижче по сторінках звіту або просто запусти `Z:\run_lab5_auto.bat` у Windows.

1 Титульна, Мета та Дерево каталогів C:\source_OS (Варіант 6)

Сторінки 1–4 звіту · Розділи 1–2 · Рисунки 1–4

🎙 ЩО КАЗАТИ ВИКЛАДАЧУ (СЦЕНАРІЙ ДОПОВІДІ)

Добрий день! Представляю лабораторну роботу № 5 «Створення об'єктів файлової системи в командному рядку» (мій варіант — **№ 6**, отже `N = 6`, `N + 1 = 7`, `N - 1 = 5`).

Робота виконувалась у Windows 11 на системному томі NTFS у консолі `cmd.exe`, запущеній від імені адміністратора. На першому етапі (рисунок 1) я створив у корені диска `C:\` каталог `source_OS` з трьома підкаталогами: `labs`, `lects` та `.hidden`. Командою `attrib +H .hidden` я зробив каталог `.hidden` прихованим: звичайний `dir` його не бачить, а `dir /AH` виводить тільки його.

Далі (рисунок 2) оператором перенаправлення `echo lab6 > labs\lab6.txt` та `echo lab7 > labs\lab7.txt` я створив текстові файли в каталозі `labs`. У каталозі `lects` (рисунок 3) командою `copy con` із натисканням комбінації кінця файлу **Ctrl+Z** створив два порожні файли (0 байт) — `lect6.txt` і `lect5.txt`.

У прихованому каталозі `.hidden` (рисунок 4) записав ім'я поточного користувача зі змінної середовища `%USERNAME%` (`Taras`) у файл `user6.txt`, а графічне дерево каталогу `tree /f /a .` — у файл `tree6.txt`.

📊 ПОЯСНЕННЯ РЕЗУЛЬТАТІВ (ЧОМУ ТАК ВИЙШЛО)

Пояснення результатів (Рис. 1–4): Навіть якщо каталог має атрибут `+H` (Hidden), файлова система не забороняє доступ до нього: якщо користувач точно знає ім'я каталогу, він може вільно зайти в нього командою `cd .hidden` або звернутися до файлів за прямим шляхом.

👉 ІНСТРУКЦІЯ ДЛЯ ТЕБЕ (ЩО НАТИСНУТИ / ПОКАЗАТИ)

Що показати: На рис. 1 — порівняння `dir` та `dir /AH`; на рис. 3 — розмір `0 bytes` біля `lect5.txt` і `lect6.txt`; на рис. 4 — вміст `tree6.txt`.

5 Каталог OS: посилання SYMLINKD, JUNCTION, HARDLINK та файл result6.txt

Сторінки 5–7 звіту · Розділи 3–4 · Рисунки 5–9

🎙 ЩО КАЗАТИ ВИКЛАДАЧУ (СЦЕНАРІЙ ДОПОВІДІ)

У домашньому каталозі користувача (`%USERPROFILE% = C:\Users\Taras`) я створив каталог `OS` та підкаталог `OS\links` (рисунок 5). Всередині `OS` я створив два типи посилань на каталоги:

• `mklink /d lects C:\source_OS\lects` — символічний зв'язок на каталог (**SYMLINKD**);

• `mklink /j labs C:\source_OS\labs` — точка з'єднання (**JUNCTION**).

На рисунку 6 видно, що `dir` позначає їх тегами `<JUNCTION>` і `<SYMLINKD>` з цільовим шляхом у квадратних дужках, а команда `tree /f` проходить крізь них прозоро, показуючи файли `lab6.txt`, `lab7.txt`, `lect5.txt` і `lect6.txt`.

Далі (рисунок 7) я сформував файл `result6.txt`: перенаправленням `>` записав туди вміст `user6.txt`, а оператором дописування `>>` додав номер варіанта `6`, два порожні рядки командою `echo.` і вміст файлів `labs\lab6.txt` та `labs\lab7.txt`.

На рисунку 8 командою `mklink /h links\tree6_h.txt C:\source_OS\.hidden\tree6.txt` я створив **жорсткий зв'язок (Hard Link)** на файл. Команда `fsutil hardlink list` показала обидва рівноправні шляхи до цього файлу на томі NTFS, і цей список я також дописав у `result6.txt`, обрамивши двома порожніми рядками (рисунок 9).

📊 ПОЯСНЕННЯ РЕЗУЛЬТАТІВ (ЧОМУ ТАК ВИЙШЛО)

Пояснення результатів (Рис. 5–9): Чому у виводі `dir` біля `tree6_h.txt` немає жодної позначки на кшталт `<HARDLINK>`, а просто пишеться звичайний файл розміром 283 байти? Тому що **Hard Link** — це не окремий об'єкт-посилання (Reparse Point), а другий запис імені в каталозі, який посилається на той самий номер файлового запису (MFT Record Number / Inode) у таблиці MFT. Обидва імена абсолютно рівноправні!

👉 ІНСТРУКЦІЯ ДЛЯ ТЕБЕ (ЩО НАТИСНУТИ / ПОКАЗАТИ)

Що показати: На рис. 6 — рядки `<JUNCTION> labs` та `<SYMLINKD> lects`; на рис. 9 — підсумковий вміст `result6.txt` (Taras, 6, два відступи, lab6, lab7, два відступи, два шляхи жорсткого зв'язку).

8 Точки монтування (mountvol), diskpart та Перевірка обмежень FAT32 і NTFS

Сторінки 8–10 звіту · Розділ 5 · Рисунки 10–15

🎙️ ЩО КАЗАТИ ВИКЛАДАЧУ (СЦЕНАРІЙ ДОПОВІДІ)

На рисунку 10 я зберіг дерево каталогу `OS` у файл `tree_OS.txt`, а на рисунку 11 створив порожні каталоги `C:\mount\NTFS_6` та `C:\mount\FAT_6` як точки монтування розділів.

Щоб перевірити монтування на практиці, я підключив до ВМ другий віртуальний диск на 2 ГБ і через утиліту `diskpart` створив на ньому два основні розділи без літер дисків: перший на 1000 МБ у форматі **NTFS** (мітка `NTFS_6`), другий на решту місця у форматі **FAT32** (мітка `FAT_6`) — рисунок 12.

На рисунку 13 командою `mountvol` я змонтував обидва томи в порожні папки `C:\mount\NTFS_6` та `C:\mount\FAT_6` за їх унікальними ідентифікаторами `\?\Volume{GUID}\`. Тепер у `dir C:\mount` вони відображаються як `<JUNCTION>` на відповідні томи, і те саме видно у графічній оснастці «Керування дисками» (рисунок 14).

Нарешті, на рисунку 15 я експериментально довів обмеження файлових систем:
1) На томі **FAT32** (`C:\mount\FAT_6`) неможливо створити ні жорсткий зв'язок (помилка `Incorrect function`), ні символічний зв'язок (`The device does not support symbolic links`).
2) Між **різними томами NTFS** (між `C:` і змонтованим томом `NTFS_6`) неможливо створити жорсткий зв'язок (`The system cannot move the file to a different disk drive`), але всередині самого тому `NTFS_6` жорсткий зв'язок створюється успішно.
3) А от **символічний зв'язок** (`mklink` без ключів) спокійно створюється на томі `NTFS_6` і веде на файл іншого тому `C:\source_OS\.hidden\tree6.txt`!

📊 ПОЯСНЕННЯ РЕЗУЛЬТАТІВ (ЧОМУ ТАК ВИЙШЛО)

Пояснення результатів (Рис. 12–15): Чому жорсткий зв'язок (Hard Link) не може перетинати межу томів навіть якщо обидва томи відформатовані в NTFS? Тому що кожен том NTFS має **власну незалежну таблицю MFT** (Master File Table) зі своєю нумерацією файлових записів, тому запис у каталозі одного тому не може посилатися на номер MFT-запису іншого тому.

👉 ІНСТРУКЦІЯ ДЛЯ ТЕБЕ (ЩО НАТИСНУТИ / ПОКАЗАТИ)

Що показати: На рис. 13 — вивід `mountvol` та `<JUNCTION> FAT_6 / NTFS_6`; на рис. 15 — 3 повідомлення про помилки та успішний `symbolic link created` між томами.

? Висновки та Контрольні питання (ЛР 5)

Сторінки 11–13 звіту · Таблиця 1 · 6 питань самоперевірки

ЩО КАЗАТИ ВИКЛАДАЧУ (СЦЕНАРІЙ ДОПОВІДІ)

Висновки: За допомогою утиліт командного рядка Windows (`mkdir` , `attrib` , `copy con` , `tree` , `mklink` , `fsutil` , `diskpart` , `mountvol`) створено і досліджено всі види об'єктів ФС для варіанта 6. Практично доведено, що з'єднання (Junction), символічні (Symlink) і жорсткі зв'язки (Hard Link) та точки монтування підтримуються виключно у файловій системі **NTFS** і відсутні у **FAT32**. При цьому Hard Link працює лише для файлів у межах одного тому, а Symlink може вказувати як на файли, так і на каталоги на інших томах. Дякую, готовий відповісти на питання!

1. Відмінність між базовим та динамічним дисками в ОС Windows. +

2. Відмінність між поняттями «розділ» (partition) та «том» (volume) у ОС Windows. +

3. Яка файлова система в ОС Windows надає можливість використовувати її папки для монтування томів? +

4. Яка відмінність у монтуванні томів за допомогою команд mountvol та mklink? +

5. В чому переваги використання UID (GUID) тому, а не його літерного позначення? +

6. У чому різниця між властивостями жорстких (Hard Link) та символічних зв’язків (Symbolic Link), з'єднань (Junction)? Для яких ФС можна використати зв'язування? +

Підготовка на Mac та Контекст роботи (STEM-проєкт)

8 C-програм у папці STEM PROJECT/код — як зібрати і запустити

🚀 ШПАРГАЛКА ПІДГОТОВКИ ТА КОНТЕКСТ (ДЛЯ ТЕБЕ НА MAC)

Контекст того, як зроблена робота: Спільний дослідницький STEM-проєкт (Загорський Тарас, Новосел Назар, Боровиков Іван, ІПЗ-31). Досліджено проблему стану гонитви (**Race Condition**) на двох архітектурах: **ARM64** (Apple M1 Pro, 4 ядра) та **x86-64** (Intel Xeon, 2 ядра). У папці `STEM PROJECT/код/` лежать 8 програм мовою C (стандарт C11, бібліотека `pthread` та `<stdatomic.h>`), `Makefile` і скрипт `run_experiments.sh`.

- **1. Автоматичний запуск на Mac одною командою в Терміналі:**
  `cd "/Users/taras/Desktop/unik/Operation Systme" && ./start_defense.sh 6`
  Відкриє PDF-звіт STEM-проєкту, збере всі C-програми і виконає живий демонстраційний прогон!
- **2. Де лежать готові CSV-таблиці та графіки:**
  - Усі CSV-результати понад 1100 запусків: `STEM PROJECT/результати/aarch64/` та `STEM PROJECT/результати/x86_64/`
  - Графіки: `STEM PROJECT/графіки/` (`g1_rozpodil_vtrat.png` – `g8_store_buffering.png`)

💻 ПОЯСНЕННЯ КОДУ, ФУНКЦІЙ ТА ПАРАМЕТРІВ

```
cd '/Users/taras/Desktop/unik/Operation Systme' && ./start_defense.sh 6
```

Відкриває PDF STEM-проєкту та запускає живу демонстрацію C-програм.

20 Построковий розбір 20 ключових рядків коду C та команд (Шпаргалка)

Пояснення функцій pthread, stdatomic, семафорів, асемблера та аргументів CLI

ЩО КАЗАТИ ВИКЛАДАЧУ (СЦЕНАРІЙ ДОПОВІДІ)

У програмному комплексі STEM-проєкту ми реалізували 8 програм мовою C. Ось построковий розбір 20 ключових конструкцій коду, системних викликів POSIX Threads і команд запуску експериментів E1–E9.

ПОЯСНЕННЯ КОДУ, ФУНКЦІЙ ТА ПАРАМЕТРІВ

```
1. gcc -O0 -pthread race_demo.c -o race_demo
```

Компіляція `race_demo` **:** прапорець `-O0` вимикає оптимізацію компілятора, щоб операція `counter++` гарантовано транслювалася у три окремі машинні інструкції `LOAD / ADD / STORE` у пам'яті, а не згорталася у регістр; `-pthread` підключає бібліотеку потоків POSIX.

```
2. pthread_create(&tid[t], NULL, worker, NULL); pthread_join(tid[t], NULL);
```

Керування потоками: `pthread_create` створює потік ядра ОС, що виконує функцію `worker` ; `pthread_join` блокує головний потік `main` до завершення всіх робочих потоків.

```
3. pthread_mutex_lock(&lock); counter++; pthread_mutex_unlock(&lock);
```

Захист критичної секції м'ютексом (`race_demo.c` **):** `pthread_mutex_lock` атомарно захоплює замок (якщо він зайнятий — переводить потік у стан сну через системний виклик ядра `futex`), після інкременту `pthread_mutex_unlock` звільняє замок і будить чергу.

```
4. ./race_demo 4 1000000 та ./race_demo 4 1000000 --mutex
```

Запуск демонстрації гонитви: параметри `4` (кількість потоків), `1000000` (ітерацій на потік, очікувано 4 000 000). Без захисту втрачається ~65–71% інкрементів; з прапорцем `--mutex` втрачено 0 (OK).

```
5. int tmp = balance; usleep(20000); tmp += a->amount; balance = tmp;
```

Штучне розширення вікна гонитви (`race_trace.c` **):** між читанням балансу рахунку (100 грн) і записом вставлено затримку `usleep(20000)` (20 мс), щоб наочно побачити з мітками часу `clock_gettime(CLOCK_MONOTONIC)` , як два банкомати затирають результат один одного (130 грн замість 180 грн).

```
6. ARM64: ldr x8, [x0] → add x8, x8, #1 → str x8, [x0] VS ldaddal x8, x8,
[x0]
```

Дизасемблювання інкременту на ARM64 (`asm_demo.c`): звичайний `(*c)++` компілюється у 3 інструкції (читання `ldr` , додавання `add` , запис `str`), між якими потік можуть перервати. Атомарний `atomic_fetch_add` — це **одна неподільна інструкція** процесора ARMv8.1 LSE `ldaddal` .

`7. x86-64: mov (%rdi), %rax → add $1, %rax → mov %rax, (%rdi) VS lock add q $1, (%rdi)`

Дизасемблювання на x86-64: атомарний інкремент використовує апаратний префікс шини/кешу `lock addq` , який утримує кеш-лінію процесора у монопольному стані протоколу когерентності MESI до завершення операції читання-модифікації-запису.

`8. pthread_spin_lock(&spin); counter++; pthread_spin_unlock(&spin);`

Спін-блокування (`sync_bench.c` , режим `spin`): на відміну від м'ютекса, потік не засинає в ядрі ОС, а крутиться в активному циклі опитування замка (busy-waiting), спалюючи процесорний час.

`9. sem_init(&sem, 0, 1); sem_wait(&sem); counter++; sem_post(&sem);`

Двійковий семафор Дейкстри (режим `sem`): `sem_init` ініціалізує семафор одиницею (`0` — спільний для потоків одного процесу); `sem_wait` (операція P) зменшує лічильник з 1 до 0 (або блокує потік, якщо вже 0); `sem_post` (операція V) повертає 1.

`10. atomic_fetch_add(&counter_a, 1);`

Атомарна операція C11 (режим `atomic`): виконує апаратний атомарний інкремент змінної типу `atomic_long` без жодних викликів ядра ОС (швидше за м'ютекс у 2–4 рази).

`11. volatile long local = 0; for(...) local++; pthread_mutex_lock(&mtx); counter += local; pthread_mutex_unlock(&mtx);`

Локальне накопичення (режим `local`): кожен потік рахує у своїй приватній стек-змінній `local` без конкуренції за кеш-лінію, і лише 1 раз у кінці додає суму під м'ютексом (найшвидший спосіб — **2,4 мс** проти 238 мс у м'ютекса!).

`12. ./sync_bench <none|mutex|spin|sem|atomic|local> 4 1000000 [--csv]`

Запуск бенчмарку синхронізації (серії E1–E5): тестує вказаний механізм на заданій кількості потоків та ітерацій; ключ `--csv` виводить рядок для автоматичного запису у таблицю.

`13. sem_init(&empty, 0, N); sem_init(&full, 0, 0); // Буфер на N=8 комірок`

Задача «Виробник–Споживач» (`prodcons.c` , серія E7): лічильний семафор `empty = 8` рахує кількість вільних місць у кільцевому буфері, а `full = 0` — кількість заповнених елементів.

`14. put(): sem_wait(&empty); pthread_mutex_lock(&mtx); buffer[in]=item; in=(in+1)%N; pthread_mutex_unlock(&mtx); sem_post(&full);`

Алгоритм виробника (`prodcons.c`): спочатку чекає вільну комірку (`sem_wait(&empty)`), потім під м'ютексом кладе елемент у буфер і сигналізує споживачам про появу даних (`sem_post(&full)`). У режимі `none` втрачається до 47% даних, у режимі `sem` — 0% втрат.

`15. case 2 (fine_naive): pthread_mutex_lock(&acc_lock[from]); pthread_mutex_lock(&acc_lock[to]);`

Взаємне блокування / Deadlock у моделі банку (`bank.c` , серія E8): коли Потік 1 переказує з рахунку A на B (бере замок A і чекає B), а Потік 2 одночасно переказує з B на A (бере замок B і чекає A), виникає циклічне очікування Коффмана — програма зависає у 10 з 10 запусків (фіксується сторожовим потоком watchdog через 1 с).

`16. case 3 (fine_ordered): int lo = from < to ? from : to, hi = from < to ? to : from; lock(lo); lock(hi);`

Усунення Deadlock впорядкуванням ресурсів (`bank.c`): ми порушуємо 4-ту умову Коффмана (циклічне очікування), завжди захоплюючи м'ютекси рахунків строго в порядку зростання їх індексу (`lo` → `hi`). Результат: 0 взаємних блокувань і точне збереження суми банку 10 000 грн!

`17. flag_p[me] = 1; turn_p = other; while (flag_p[other] && turn_p == other) ;`

Класичний алгоритм Петерсона (`peterson.c` , серія E9): на сучасних багатоядерних процесорах у режимі `plain` (звичайні `volatile` змінні) взаємне виключення **ламається у 100% запусків** через апаратну перестановку Store→Load у буфері записів (Store Buffer).

`18. atomic_store(&flag_a[me], 1); atomic_store(&turn_a, other); // memory_order_seq_cst`

Виправлений алгоритм Петерсона (режим `seq`): використання атомарних операцій з повною послідовною узгодженістю `memory_order_seq_cst` вставляє бар'єри пам'яті і дає **0 порушень** критичної секції.

`19. Потік 1: X = 1; barrier(); r1 = Y; | Потік 2: Y = 1; barrier(); r2 = X;`

Лакмус-тест Store Buffering (`litmus_sb.c`): перевіряє, чи можуть обидва потоки одночасно прочитати старі нулі (`r1 == 0 && r2 == 0`). Без апаратного бар'єру (`none`) цей «неможливий» результат виникає у **8,4%** ітерацій на ARM64 та у **27,1%** на x86-64!

`20. atomic_thread_fence(memory_order_seq_cst); // ARM64: dmb ish, x86: mfence`

Повний апаратний бар'єр пам'яті (`litmus_sb fence`): змушує процесор спочатку скинути буфер записів (Store Buffer) у кеш L1 до виконання наступного читання. Результат: **0 разів** `r1 = r2 = 0` з 1 000 000 ітерацій.

📊 ПОЯСНЕННЯ РЕЗУЛЬТАТІВ (ЧОМУ ТАК ВИЙШЛО)

Підсумок по коду STEM: Усі 8 програм наочно доводять 4 гіпотези роботи та демонструють поведінку процесорів ARM64 та x86-64 на рівні асемблера та когерентності кешу.

👉 ІНСТРУКЦІЯ ДЛЯ ТЕБЕ (ЩО НАТИСНУТИ / ПОКАЗАТИ)

Використовуй цю вкладку, якщо також захищатимеш або згадуватимеш STEM-проєкт.